API ADRs

NHS Pathways Transformation

5 November, 2026

Table of Contents

1 ADR-API-001: AWS Account for POCs 12
1.1 Context 12
1.2 Decision 12
1.2.1 Assumptions 12
1.2.2 Options 13
1.2.3 Recommendation 14
1.3 Consequences 14
1.4 Compliance 14
1.5 Actions 14
2 ADR-API-002: Triage API hosting options 15
2.1 Context 15
2.2 Decision 16
2.2.1 Recommendation 16
2.2.2 Rationale 16
2.2.3 Assumptions 17
2.2.4 Drivers 18
2.2.5 Criteria 18
2.2.6 Options 19
2.2.7 Option 1: Single Region, Multi-AZ 20
2.2.8 Option 2: Multi-Region Active/Active 21
2.2.9 Option 3: Single Region, Multi-AZ with Second Region DR 22
2.2.10 Option A: Direct Connect Add-On (for Options 1–3) 23
2.2.11 Option 4: Outpost – Local Compute 24
2.2.12 Option 5: Containerised Local Deployment 24
2.2.13 Option 6: Multi Cloud hosting and deployment 25
2.3 Consequences 25
2.4 Compliance 25
2.5 Notes 26
2.6 Actions 26

3 ADR-API-003: Triage API communication format options 27
3.1 Context 27
3.2 Decision 27
3.2.1 Assumptions 27
3.2.2 Drivers 28
3.2.3 Criteria 28
3.2.4 Options 29
3.2.5 Recommendation 30
3.2.6 Rationale 30
3.3 Consequences 31
3.4 Compliance 31
3.5 Notes 32
3.6 Actions 32
4 ADR-API-004: Triage API integration with other NHS APIs 33
4.1 Context 33
4.2 Decision 34
4.2.1 Assumptions 34
4.2.2 Drivers 35
4.2.3 Criteria 35
4.2.4 Options 35
4.2.5 Recommendation 35
4.2.6 Rationale 36
4.3 Consequences 37
4.4 Compliance 37
4.5 Notes 38
4.6 Actions 38
5 ADR-API-005: Choice of Triage API PoC development language 39
5.1 Context 39
5.2 Decision 40
5.2.1 Assumptions 40
5.2.2 Drivers 40
5.2.3 Criteria 40

5.2.4 Options 40
5.2.5 Recommendation 41
5.2.6 Rationale 41
5.3 Consequences 41
5.4 Compliance 42
5.5 Notes 42
5.6 Actions 43
6 ADR-API-006: Triage API Architecture 44
6.1 1. Context 44
6.2 2. Business Drivers 44
6.3 3. Decision 45
6.3.1 3.1 Options 45
6.3.2 3.2 Recommendation 46
6.4 4. Consequences 46
6.5 5. Compliance 46
6.6 6. Actions 46
6.6.1 Stateless flow diagram 47
7 ADR-API-007: Triage API Versioning 48
7.1 1. Context 48
7.2 2. Business Drivers 49
7.3 3. Decision 49
7.3.1 3.1 Options 49
7.3.2 3.2 Recommendation 51
7.4 4. Consequences 52
7.5 5. Compliance 52
7.6 6. Actions 52
8 ADR-API-008: Pathways Content Versioning 53
8.1 1. Context 53
8.2 2. Business Drivers 53
8.3 3. Decision 53
8.3.1 3.1 Options 53
8.3.2 3.2 Recommendation 54

8.4 4. Consequences 55
8.5 5. Compliance 55
8.6 6. Actions 55
9 ADR-API-009: [WIP]Choice of Triage API Authentication Pattern 56
9.1 WIP 56
9.1.1 Context 57
9.1.2 Decision 58
9.1.3 Consequences 60
9.1.4 Compliance 60
9.1.5 Notes 60
9.1.6 Actions 60
10 ADR-API-010: AWS Infrastructure and API Security 61
10.1 1. Context 61
10.2 2. Business Drivers 61
10.3 3. Decision 62
10.3.1 3.1 Options 62
10.3.2 3.2 Recommendation 62
10.4 4. Consequences 64
10.5 5. Compliance 64
10.6 6. Actions 64
11 ADR-API-011: Database Choice 65
11.1 1. Context 65
11.2 2. Business Drivers 65
11.3 3. Decision 66
11.3.1 3.1 Options 66
11.3.2 3.2 Recommendation 67
11.4 4. Consequences 67
11.5 5. Compliance 67
11.6 6. Actions 67
12 ADR-API-012: Triage API Data Model 68
12.1 1. Context 68
12.2 2. Business Drivers 69

12.3 3. Decision 69
12.3.1 3.1 Options 69
12.3.2 3.2 Recommendation 70
12.4 4. Consequences 74
12.5 5. Compliance 74
12.6 6. Actions 75
13 ADR-API-013: Loading Pathways into the Triage Database 76
13.1 Context 76
13.2 Business Drivers 77
13.3 Decision 77
13.3.1 Assumptions 77
13.3.2 Drivers 77
13.3.3 Criteria 77
13.3.4 Options 78
13.3.5 Recommendation (Tactical and Strategic) 81
13.3.6 Rationale 82
13.3.7 Tactical Mitigation: Manual CSV Placement Process 82
13.4 Consequences 83
13.5 Compliance 83
13.6 Notes 83
13.7 Actions 84
14 ADR-API-014: Monitoring and Observability Tooling 85
14.1 1. Context 85
14.2 2. Business Drivers 85
14.3 3. Decision 86
14.3.1 3.1 Options 86
14.3.2 3.2 Recommendation 86
14.4 4. Consequences 87
14.5 5. Compliance 87
14.6 6. Actions 87
15 ADR-API-015: Alerts 88
15.1 1. Context 88

15.2 2. Business Drivers 88
15.3 3. Decision 88
15.3.1 3.1 Options 88
15.3.2 3.2 Recommendation 89
15.4 4. Consequences 90
15.5 5. Compliance 90
15.6 6. Actions 90
16 ADR-API-016: Metrics 91
16.1 1. Context 91
16.2 2. Business Drivers 91
16.3 3. Decision 92
16.3.1 3.1 Options 92
16.3.2 3.2 Recommendation 92
16.4 4. Consequences 92
16.5 5. Compliance 93
16.6 6. Actions 93
17 ADR-API-017: Triage API - Release Process 94
17.1 1. Context 94
17.2 2. Business Drivers 94
17.3 3. Decision 95
17.3.1 3.1 Options 95
17.3.2 3.2 Recommendation 95
17.3.3 3.3 Technical Release Pipeline 97
17.3.4 3.4 Clinical Content Release Pipeline 97
17.3.5 3.5 Release Governance and Assurance 98
17.4 4. Consequences 98
17.5 5. Compliance 98
17.6 6. Actions 98
17.7 7. References 98
17.8 Triage API - Release Governance 99
17.8.1 Release Governance 99
17.8.2 Break Glass Process 99

18 ADR-API-018: Disaster Recovery and Business Continuity 100
18.1 1. Context 100
18.2 2. Business Drivers 100
18.3 3. Decision 101
18.3.1 3.1 Disaster Recovery Strategy Overview 101
18.3.2 3.2 Options 101
18.3.3 3.3 Recommendation 102
18.3.4 3.4 Business-Continuity Procedures 102
18.4 5. Compliance 103
18.5 6. Actions 103
19 ADR-API-019: Infrastructure and role permissions 105
19.1 Context 105
19.2 Drivers 105
19.3 Decision 105
19.3.1 Decision details 106
19.4 Options considered 110
19.5 Rationale 110
19.6 Consequences 110
19.7 References 111
20 ADR-API-020 - WAF Rules configuration 112
20.1 Drivers 112
20.2 Assumptions 112
20.3 Decision 112
20.4 Options considered 113
20.5 Rationale 116
20.5.1 Criteria 116
20.5.2 Scoring 116
20.5.3 Refenerence 117
21 ADR-API-021 - Clinical Release Ingestion Shared File Transfer 118
21.1 1 Context 118
21.2 1.1 Options 118
21.2.1 1.1.1 Option 1 - Retain the Current Manual Transfer Approach 118

21.2.2 1.1.2 Option 2 - Shared Pathways Release Service and Landing Bucket (RECOMMENDED) .. 119
21.2.3 1.1.3. Option 3 - Patient Triage API-Owned S3 Bucket 120
21.2.4 1.1.4. Option 4 - Consume a PAT API 121
21.2.5 1.1.4 Decision 121
21.2.6 1.1.5 Consequences 121
21.2.7 References 121
22 ADR-API-022: Performance testing with Jmeter 122
22.1 Context 122
22.2 Decision 122
22.3 Decision Drivers 123
22.4 Considered Options 123
22.4.1 Option 1: ECS on EC2 123
22.4.2 Option 2: ECS Fargate (chosen) 123
22.4.3 Option 3: EKS 124
22.4.4 Option 4: External platform 124
22.4.5 Option 5: Local/self-hosted runners 124
22.5 Consequences 125
22.6 Architecture (High-Level) 125
22.7 Security and Compliance 125
22.8 Cost Considerations 125
22.9 Implementation Plan 126
22.10 Acceptance Criteria 126
23 ADR-API-023: Green-Blue Deployment Options 127
23.1 Context 127
23.1.1 Constraints and requirements: 127
23.2 Decision Drivers 128
23.2.1 Key design points: 128
23.3 Considered Options 128
23.3.1 Option 1: API Gateway Routing Rule Amendment (Proposed) 128
23.3.2 Option 2: Application Load Balancer (ALB) with Weighted Target Groups 129
23.3.3 Option 3: API Gateway Stages for Green/Blue 130
23.3.4 Option 4: Route53 Weighted Routing Records 130

23.3.5 Option 5: Routing Lambda 131
23.4 Comparison Matrix 132
23.5 Decision 133
23.6 Consequences 133
23.6.1 Positive: 133
23.6.2 Negative: 133
23.7 Architecture (High-Level) 134
23.8 Security and Compliance 135
23.8.1 mTLS Termination at API Gateway 135
23.8.2 Data in Transit 136
23.9 Cost Considerations 136
23.9.1 Proposed Option i.e. API Gateway Routing Rule Amendment 136
23.9.2 Rejected Options Cost Comparison 137
23.9.3 Cost Optimisation 138
23.10 Implementation Plan 138
23.10.1 Pre-Switchover 138
23.10.2 Switchover 139
23.10.3 Post-Switchover 139
23.10.4 Rollback 139

- ADR-API-001: AWS Account for POCs (see page 12)
- ADR-API-002: Triage API hosting options (see page 15)
- ADR-API-003: Triage API communication format options (see page 27)
- ADR-API-004: Triage API integration with other NHS APIs (see page 33)
- ADR-API-005: Choice of Triage API PoC development language (see page 39)
- ADR-API-006: Triage API Architecture (see page 44)
- ADR-API-007: Triage API Versioning (see page 48)
- ADR-API-008: Pathways Content Versioning (see page 53)
- ADR-API-009: [WIP]Choice of Triage API Authentication Pattern (see page 56)
- ADR-API-010: AWS Infrastructure and API Security (see page 61)
- ADR-API-011: Database Choice (see page 65)
- ADR-API-012: Triage API Data Model (see page 68)
- ADR-API-013: Loading Pathways into the Triage Database (see page 76)
- ADR-API-014: Monitoring and Observability Tooling (see page 85)
- ADR-API-015: Alerts (see page 88)
- ADR-API-016: Metrics (see page 91)
- ADR-API-017: Triage API - Release Process (see page 94)
    - Triage API - Release Governance (see page 99)
- ADR-API-018: Disaster Recovery and Business Continuity (see page 100)
- ADR-API-019: Infrastructure and role permissions (see page 105)
- ADR-API-020 - WAF Rules configuration (see page 112)
- ADR-API-021 - Clinical Release Ingestion Shared File Transfer (see page 118)
- ADR-API-022: Performance testing with Jmeter (see page 122)
- ADR-API-023: Green-Blue Deployment Options (see page 127)

1 ADR-API-001: AWS Account for POCs

Item	Example
Date	11 Feb 2025
Status	`Proposed`
Deciders	`Architecture and Engineering`
Significant	
Owners	SaET Team
Decision	
Decision waiver	

1.1 Context

The Pathways or Safe and Effective Triage (SaET) team are in a phase where we are conducting multiple POCs in the upcoming sprints. A few of these POCs would require a cloud environment to deploy and carry out tests which would help set the future direction. It has been previously agreed that we use AWS for the POCs however within AWS there are multiple AWS account. This page documents the options and provides a recommendation to move forward with the environments

1.2 Decision

1.2.1 Assumptions

Listed below are some of the assumptions

- AWS would be used for the POC's
- Within AWS we would be using the NHSE managed AWS account.

1.2.2 Options

1.2.2.1 Option 1 - Use existing AWS account owned by the Pathways/SaET BAU team

In this option, we would be using an existing AWS account owned by the Pathways/SaET BAU team.

Pros	Cons
Account already present, hence no additional requests needed	Does not currently have a Dev account for us to use
Ability to share any common features like AWS Security services, monitoring solutions etc	Cost management be complicated
Cost benefit from sharing common services or solutions	Managing deployments through terraform would involve coordination
	Tidying up the account after the POCs completes would be complicated
	BAU team in the past faced an additional task of removing resources left by the previous supplier

1.2.2.2 Option 2 - Create a new AWS account owned by Pathways/SaET Transformation Team

In this option, we would be requesting a new AWS account (only development) which would be owned and managed by the Pathways/SaET team.

Pros	Cons
Transformation has its own development environment and there are no dependencies with the BAU Team	Account not already present, hence additional request needs to be raised
Cost management is simplified	It will have its own common services like AWS Security service, monitoring tool etc
Managing deployments with or without terraform is simplified	A cost overhead for having its own common solutions instead of sharing one in another account
Less friction when going live with new products	

Pros	Cons
Easily retire old solutions or old accounts	
Tidying up the new account is easy	

1.2.3 Recommendation

Option 2 - Create a new AWS account owned by Pathways/SaET Transformation Team is recommended based on the pros and cons listed above.

1.3 Consequences

If approved, we would for a period of time have multiple Pathways AWS accounts for development environment only.

1.4 Compliance

Compliance to the standards specified within NHSE and other best practices/standards would be adhered to

1.5 Actions

Actions	Raised By	Raised Date	Resolved By	Resolved Date

2 ADR-API-002: Triage API hosting options

Item	Example
Date	11 Jun 2025
Status	IN PROGRESS
Deciders	Engineering, Architecture, Technical Review and Governance
Significant	Non-functional characteristics
Owners	Afzal Mufti
Decision	Submit to TRG as KAD once Alpha SDO submitted.
Decision waiver	conditions placed on the approval (e.g. approved as a tactical solution, strategic solution to be implemented by <date>, etc.)

2.1 Context

The Pathways triage API is a new digital service intended to facilitate real-time patient triage assessment at national scale: 100 TPS within 200-500ms response latency targets.

Services that are intended to consume this API for live triage are: Ambulance 999, 111 Telephony, 111 Online, Emergency Department Streaming & Redirect. Future care settings and channels may emerge in future.

The hosting model must ensure high availability, resilience, cost-effectiveness and alignment with:

- NHS England Architecture principles[1]
- NHS England Solution Architecture Framework[2]
- NHS England Engineering red lines[3]

This decision is related to existing NHS national service patterns (e.g., Spine, PDS, 111 Online) which typically run in a single region using Multi-AZ for high availability.

Constraints include cost, complexity and maintainability.

Options:

- 1. Single region multi AZ

1. https://digital.nhs.uk/developer/architecture/principles
2. https://architecture.digital.nhs.uk/solution-architecture-framework
3. https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/Red-lines.aspx

- 2. Multi region Active / Active
- 3. Single region multi AZ with 2nd region DR deployment
- a. Direct connect add-on (options 1,2,3)
- 4. Outpost local deployed compute
- 5. Containerised local deployment
- 6. Multi cloud hosting and deployment

2.2 Decision

2.2.1 Recommendation

The recommendation is to choose Option 1 (Single Region, Multi-AZ) at least initially.

- Take a phased approach to hosting the Pathways Triage API
- Start with Option 1 - Single region, Multi-AZ
- Evolve to Option 2 - Multi region Active / Active
  - Subject to hosting policy exception and local connectivity concerns
- Utilise direct connect as an additional extra depending on connectivity requirements at the Healthcare service e.g. 999 provider sites.

2.2.2 Rationale

Business / EA view

- Choice of option 1 is based on the NFRs that we know now (June 2025). A key one being that BC considerations are out of scope for the delivery of the Triage API.
- Option 1 aligns with and is consistent with existing NHS platinum national services (e.g., Spine, PDS) which typically run in a single region using Multi-AZ for high availability.
- Other more resilient patterns exist (e.g., Multi-region Active/Active, DR), however complexity, maturity, and cost must be taken in to consideration.
- Option 1 is Cost-effective and can extend to Option 2 as the service transitions in maturity

Run/maintain view

- Options 1-3 are a common operating pattern, relatively low complexity to the other options considered. NHSE Organisation best practice and common ways of working can be leveraged here.
- Cloud based options (1-3) are largely interchangeable / reversible. Resilience can be increased based on cost/risk appetite
- Options 4 and 5 will involve SLA contracts and deployed hardware not easily reversible. Containerised approach also has impact on the whole system architecture not easily reversible.

- A cloud based solution allows us to leverage cloud native and serverless technologies to deliver a performant and cost effective solution. It also allows us to focus on engineering excellence and less on the complexity of hosting and local deployments

2.2.3 Assumptions

- Some provider business continuity (BC) plans may include local implementations of triage logic or data, this is considered out of scope for the national triage API delivery and remains the responsibility of the provider.
- The Triage API service will operate as Platinum level service.
  - Availability 99.9%
  - DR recovery target of 2 hours
  - Service Classifications[4]
- A cloud-native AWS architecture will be used
  - [ ] TODO Requires a separate ADR
- Service resilience expectations will grow and risk appetite will decrease over time as the service matures through private beta, public beta, live
- Hosting costs must remain controlled in early phases
- Operational teams are capable of running and maintaining Multi-AZ AWS deployments. (This is built in to core serverless services e.g. API Gateway, Lambda, S3, Dynamo)
- Local (on-site) disaster recovery mitigations are out of scope of the API design
- The volumetrics below were used to calculate the TPS, based on 2024/25 data
  - TPS was calculated as follows: (Peak per day * Questions per triage) / 86400
- The API will experience x TPS at peak and be provisioned for + 20% - Needs qualifying. Update: Nov '25. We have more detailed data to more accurately model expected traffic. will update this page in due course.

Service	Triages started (2024/25)	Average number of triages per day	Peak number of triages per day	Average number of questions per triage	Average number of API requests per call	Transactions per second (TPS)
Ambulance 999	5,099,768	13,972	17,003	19	19	4
111 Telephony	19,514,394	53,464	93,270	20	20	22

4. https://digital.nhs.uk/services/cloud-centre-of-excellence/cloud-security-good-practice-guide/9.-appendix-b-service-classifications

Service	Triages started (2024/25)	Average number of triages per day	Peak number of triages per day	Average number of questions per triage	Average number of API requests per call	Transactions per second (TPS)
111 Online	7,794,040	21,354	36,549	11	11	5
Emergency Department Streaming & Redirection	358,579	982	1,470	17	17	1
Total	32,766,781	89,772	148292			32

2.2.4 Drivers

- Key non-functional requirements identified through internal and external stakeholder engagement
  - Low latency e.g. within 300ms response, similar to Spine
  - National load e.g. 30 TPS at peak
  - Operational simplicity at the national side
  - Low complexity at the host system supplier side
  - Scalability according to traffic demand
  - Cost optimised

2.2.5 Criteria

- Performance
- Resilience
- Low operational complexity
- Cost efficiency
- Scalability
- Alignment to NHSE strategy and guidance

2.2.6 Options

Option	Performance	Resilience	Low op complexity	Cost efficient	Scalability	Strategic Alignment	Score	Notes
1. Single region multi AZ	5	3	5	5	5	5	28	Aligns with other platinum services. Vulnerable to a region outage.
2. Multi region Active / Active (Same cloud provider)	5	5	4	4	5	4	27	Potentially over engineering to meet the resilience NFR. No other services with this pattern.
3. Single region multi AZ with 2nd region DR deployment	5	5	3	3	5	3	24	Complexity increases to enable multi region auto deployment. Some risk if not dual running from the DR site.
a. Direct connect add-on (options 1,2,3)		+1		-1				Optional extra where warranted for additional resilience e.g. 999 provider sites. Cost incurred by provider not centrally.
4. Outpost local deployed compute	5	5	2	1	2	1	16	Engineering complexity, not cloud first. Danger of overprovisioning for peak traffic. Costly.
5. Containerised local deployment	5	5	1	1	1	1	14	Engineering complexity, not cloud first. Danger of overprovisioning for peak traffic. However, would put control in the hands of providers that want to own that responsibility.

Option	Performance	Resilience	Low op complexity	Cost efficient	Scalability	Strategic Alignment	Score	Notes
7. Multi Cloud hosting and deployment	5	5	1	1	5	1	18	Very resilient but complex to build/run/ maintain and expensive. Lots of duplication. Risks to dual run deployments.

2.2.7 Option 1: Single Region, Multi-AZ

Deploy the service within a single AWS region (London) across multiple Availability Zones (AZs) for high availability.

- This model is used by other platinum level national systems such as PDS and Spine.
- Pros:
  - Simpler to deploy and operate.
  - Well understood and common pattern
  - Cost-effective.
  - Resilient to AZ-level failures.
- Cons:
  - Still vulnerable to a full region outages.
    Note: in the event of a London region outage then Spine and other national systems will be affected. Additionally, providers and suppliers themselves who have cloud hosting in the affected region would also be impacted.
  - Does not enable a like for like offline mode

For additional cost, this option could take advantage of AWS direct connect - establishing a dedicated private network connection between an on-premise network and AWS bypassing the public internet. This would add resilience to the solution however does introduce extra cost, complexity and an additional point of failure.

Risk	Mitigation
Single AZ outage	Multi-AZ deployment, failover to healthy AZ for service continuity
Full region outage	No mitigation. Service impacted until region recovers. **Important note**: if this were to occur, the vast majority of healthcare services would also be impacted. Spine itself and other national services are currently hosted in the AWS London region. Provider systems hosted in London region would also be impacted.

Risk	Mitigation
Local connectivity issue	• Retry logic • Local failover to alternate connection provider, e.g. alternate line, 5G • Use of AWS direct connect dedicated private network connection
Latency	Dependent on the source and cause of the latency. The Triage API design team are currently exploring feasibility and options for front loading some data in the API response to lazy-load some questions and provide a better user experience.
DDoS	API secured using NHSE API Management platform. Use of AWS Shield, WAF, rate limits.

2.2.8 Option 2: Multi-Region Active/Active

Deploy the service in multiple AWS regions (E.g. London, Ireland) and route traffic to all sites.

- Pros:
  - Highest availability and geographic resilience.
  - Resilient to region-level failures.
- Cons:
  - Increased complexity (synchronisation, deployment).
  - Increased cost and operational overhead. However, assuming using serverless architecture this would scale proportionately to traffic.
  - Consider this over-engineered for the initial MVP.
  - Agreement to operate in a non-UK region would need an agreed policy exception (GDS multi-region guidance[5])

Risk	Mitigation
Single AZ outage	Same as option 1
Full region outage	Mutli-region deployment. Reroute traffic to active region. **Important note**: if this were to occur, the vast majority of healthcare services would also be impacted. Spine itself and other national services are currently hosted in the AWS London region. Provider systems hosted in London region would also be impacted.

5. https://www.gov.uk/government/publications/multi-region-cloud-and-software-as-a-service/multi-region-cloud-and-software-as-a-service-html

Risk	Mitigation
Local connectivity issue	• Retry logic • Local failover to alternate connection provider, e.g. alternate line, 5G • Use of AWS direct connect dedicated private network connection. Note: a direct connect is to one Region only unless multiple connections are provisioned.
Latency	Same as option 1
DDoS	Same as option 1

2.2.9 Option 3: Single Region, Multi-AZ with Second Region DR

Operate the service in a single region with a standby (warm or cold) disaster recovery deployment in another region (E.g. London, Ireland).

- Pros:
  - Improved regional resilience
  - Cost effectiveness vs Option 2 is debatable as building in a trigger for the failover would need engineering and regular testing.
- Cons:
  - Requires DR trigger/coordination and regular failover testing.
  - Not a common pattern within NHSE
  - In the event of contingency, there will be a delay before the service is operational in the new region even with full automation.
  - Agreement to operate in a non UK region could take time (cite recent GDS guidance here)

2.2.9.1 Disaster recovery expectation:

https://docs.aws.amazon.com/whitepapers/latest/disaster-recovery-workloads-on-aws/disaster-recovery-options-in-the-cloud.html[6]

- For warm standby RPO/RTO: Minutes. Cost £££
  - This could be 5-15 minutes (Core infrastructure already provisioned, standby DB ready)
- For pilot light RPO/RTO: 10s of minutes. Cost ££
  - This could be 30-90 minutes (provision some infrastructure, data restore from backup, health checks).
- Backup and restore RPO/RTO: Hours. Cost £

6. https://docs.aws.amazon.com/whitepapers/latest/disaster-recovery-workloads-on-aws/disaster-recovery-options-in-the-cloud.html#:~:text=demonstration%20of%20implementation.-,Pilot%20light,object%20storage%2C%20are%20always%20on

- This could be 2-6 hours. Provision all resources after event. Restore from backup after event.

Risk	Mitigation
Single AZ outage	Same as option 1
Full region outage	Failover to disaster recovery region. Utilise DR backups to rebuild in a new region. **Important note**: if this were to occur, the vast majority of healthcare services would also be impacted. Spine itself and other national services are currently hosted in the AWS London region. Provider systems hosted in London region would also be impacted.
Local connectivity issue	• Retry logic • Local failover to alternate connection provider, e.g. alternate line, 5G • Use of AWS direct connect dedicated private network connection. Note: a direct connect is to one Region only unless multiple connections are provisioned.
Latency	Same as option 1
DDoS	Same as option 1
Failover delay	Full automation of DR routine and provisioning. Regular DR drills.

2.2.10 Option A: Direct Connect Add-On (for Options 1–3)

For additional cost, options 1-3 could take advantage of AWS direct connect - establishing a dedicated private network connection between an on-premise network and AWS bypassing the public internet.

- Pros:
  - Improved performance, predictable network
  - Dedicated network link - could still allow traffic in the event of a cyber event where local policy dictates operating with no inbound/outbound traffic
  - Could be architected as a primary route, falling back to internet connectivity in the event of issues
- Cons:
  - Cost and setup

Could be worth exploring for sites which feel they need it e.g. Ambulance.

2.2.11 Option 4: Outpost – Local Compute

Use AWS Outposts to run the API in a local environment extending AWS infrastructure, services and APIs to on-premise

- Pros:
  - Local data processing
  - Low-latency
  - Offline capability
- Cons:
  - High cost.
  - Infrastructure and support overhead.
  - Large rack space required
  - Adds complexity - transition between circuit breaker offline and online mode requires managing

Risk	Mitigation
Full region outage	Circuit breaker contingency - continue in local isolated mode until parent region operational
Local connectivity issue	Circuit breaker contingency - continue in local isolated mode until parent region operational Queue requests back to CQI API
Hardware failure	Hardware spares kept on-site AWS Outpost support SLA

2.2.12 Option 5: Containerised Local Deployment

Run the API in containers on local infrastructure (e.g EKS Anywhere or other on-prem deployment).

- Pros:
  - Offline capability
  - Local management autonomy of infrastructure
- Cons:
  - High complexity
  - Must be deployed to local infrastructure which may be less resilient if not optimally designed or patched
  - Local operational (staffing) overhead (run/maintain, local skills required)

Risk	Mitigation
Full region outage	Circuit breaker contingency - continue in local isolated mode until parent region operational
Local connectivity issue	Circuit breaker contingency - continue in local isolated mode until parent region operational Queue requests back to CQI API
Hardware failure	Hardware spares kept on-site Local support SLA

2.2.13 Option 6: Multi Cloud hosting and deployment

Run the API in containers hosted multi-cloud (e.g. AWS, Azure, GCP) and load balance between them

- Pros:
  - Highly resilient - protects against any single cloud provider or region failure
- Cons:
  - This pattern not currently in use in any other national systems
  - Complex to build, test, manage and maintain
  - Duplicates functionality, infrastructure and cost
  - Requires service team to maintain dual knowledge of hosting providers
  - Does not mitigate local connectivity concerns

2.3 Consequences

- Host system suppliers must onboard to and conform to the Triage API specification and use the service to run a live Pathways triage
- Whole system alignment on appropriate BC/DR is needed in light of the move to a Triage API
- The Triage API must be run as a Platinum level service and have an appropriate programme and engineering team and service wrap around it

2.4 Compliance

Establish how the success is going to be measured. Once implemented, the effect might lend itself to be measured, therefore if appropriate a set of criteria for success could be established. Compliance checks of the decision can be manual or automated using a fitness function. If it is the latter this section can then specify how that fitness function would be implemented and whether there are any other changes to the codebase needed to measure this decision for compliance.

2.5 Notes

Requires:

- [ ] Produce an ADR for cloud hosting provider
- [ ] Write up tech stack options/assumptions
- [ ] More data on volumetrics?
- [ ] SLT steer

Include any links to existing epics, decisions, dependencies, risks, and policies related to this decision record. This section could also include any further links to configuration items within the project or the codebase, signposting to the areas of change.

It is important that if the decision is sub-optimal or the choice is tactical or misaligned with the strategic directions the risk related to it is identified and clearly articulated. As a result of that, the expectation is that a Tech Debt[7] *record is going to be created on the backlog.*

2.6 Actions

7. https://github.com/NHSDigital/software-engineering-quality-framework/blob/main/tech-debt.md

3 ADR-API-003: Triage API communication format options

Date	17 Jun 2025
Status	IN PROGRESS
Deciders	Engineering, Architecture, Technical Review and Governance
Significant	Non-functional characteristics
Owners	Andrew Walker
Decision	Stateless FHIR Task
Decision waiver	

3.1 Context

The Pathways triage API is a new digital service intended to facilitate real-time patient triage assessment at national scale: 100 TPS within 200-500ms response latency targets.

Services that are intended to consume this API for live triage are: Ambulance 999, 111 Telephony, 111 Online, Emergency Department Streaming & Redirect. Future care settings and channels may emerge in future.

The communication format must be able to convey all of the information needed for a Pathways triage, yield reasonably small payloads for each request and response message, be technically easy to adopt by suppliers, and satisfy various other criteria.

This decision relates to a proof-of-concept around the use of FHIR on the assumption that, as a well-known interoperability standard in healthcare settings, it would be an attractive candidate. Discussions and investigations opened up some other possibilities, resulting in a wider options analysis piece.

3.2 Decision

3.2.1 Assumptions

- The Triage API service will operate as Platinum level service.
  - Availability 99.9%
  - DR recovery target of 2 hours
  - Service Classifications[8]

- Supplier systems maintain triage state as it progresses
- It supports 100 TPS within 200-500ms response latency targets

3.2.2 Drivers

- Speed of response is critical for patient safety
- Budgets are tight
- Pathways releases are infrequent and expensive

3.2.3 Criteria

Provide a list of criteria (expected to be a least 5) used to evaluate options. These will typically include several relevant non-functional characteristics of the product/service/capability such as performance, security, operations, reliability, sustainability, costs, technology maturity, team skillset, or strategic direction.

- **Statefulness of the API**
  - the property of a service keeping track of an ongoing business process across multiple upstream messages
  - impacts Payload sizes in that stateful solutions will not need to communicate the whole triage history
  - impacts ease of rerouting of triage traffic to other hosts / regions
  - impacts ease to build and maintain
- **Payload size**
  - a heuristic for network traffic and associated costs
  - considering typical message size
  - considering growth pattern as triage progresses
- **Compliance**
  - how well the approach aligns with standards in place in NHSE
  - precedents across the pathways, DUEC, and wider healthcare ecosystems
- **Ease of onboarding**
  - how difficult it would be for suppliers to integrate using each approach
- **Rehydration of triage**
  - future consideration re: the practice of returning to a prior state by importing the data needed
  - how relatively impactful is the ability to rehydrate?
- **Payload readability**
  - for the sake of communication within the team and with stakeholders familiar with the domain

8. https://digital.nhs.uk/services/cloud-centre-of-excellence/cloud-security-good-practice-guide/9.-appendix-b-service-classifications

    - may also support debugging and RCA of incidents
- **Tooling support**
    - options where off-the-shelf libraries can be used to assist with the creation and parsing of messages are deemed favourable
- **Validation**
    - options where off-the-shelf libraries can be used to assist with the validation of messages are deemed favourable
- **Extensibility**
    - anticipating that the payloads for communication between clients and the API may need change as the service evolves over time

3.2.4 Options

FHIR			Custom
Bundle	**Task**		
ADR-API-003: Triage API communication format options (see page 29)	ADR-API-003: Triage API communication format options (see page 29)	ADR-API-003: Triage API communication format options (see page 30)	ADR-API-003: Triage API communication format options (see page 30)

3.2.4.1 FHIR Bundle

See FHIR Bundle request response approach[9]

- This approach uses FHIR's Bundle resource as the root element of both request and response payloads.
- Request bundles contain 1 Patient resource providing demographics about the patient needed to select appropriate pathways, 1 Parameter resource to define the pathways version and operator skillset, and 0..* QuestionnaireResponse resources with answers to questions already received.
- Response bundles contain 0..1 Questionnaire resources, 0..* Communication resources for care advice, and 0..1 Task resources for the triage outcome.

3.2.4.2 FHIR Task (stateful)

See FHIR Task stateful request response approach[10]

9. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1087788913/FHIR+Bundle+request+response+approach
10. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1087788924/FHIR+Task+stateful+request+response+approach

- This approach uses FHIR's Task resource as the root element for the request payloads, and a Bundle resource as the root element for the response payloads.
- Request Tasks define patient demographics and pathways version requirements, and the answer to the latest question as input items.
- Each Request only provides the new information for the triage - expecting the API to have persisted any previously submitted content.
- Response Bundles contain 1 Task resource with latest state, 0..1 Questionnaire resources, 0..* Communication resources for care advice, and 0..1 ServiceRequest resources for the triage outcome.

3.2.4.3 FHIR Task (stateless)

See FHIR Task stateless approach[11]

- This approach uses FHIR's Task resource as the root element for the request payloads, and a Bundle resource as the root element for the response payloads.
- Request Tasks define patient demographics and pathways version requirements, and answers to questions already received as input items.
- Response Bundles contain 1 Task resource with latest state, 0..1 Questionnaire resources, 0..* Communication resources for care advice, and 0..1 ServiceRequest resources for the triage outcome.

3.2.4.4 Custom JSON

See Custom JSON request response approach[12]

- This approach structures the information in a proprietary way.
- Content can be streamlined to requirements and fields named appropriately for the specific domain.

3.2.5 Recommendation

We recommend proceeding with the stateless FHIR Task approach.

As we are pre-alpha, there is room to reverse this decision.

3.2.6 Rationale

	FHIR Bundle	FHIR Task		Custom JSON
		Stateful	Stateless	

11. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1102713093/FHIR+Task+stateless+approach
12. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1087789245/Custom+JSON+request+response+approach

Statefulness	Stateless	Stateful	Stateless	Stateless
Payload	Acceptable	Acceptable	Acceptable	Smallest
Compliance	Contentious	Compliant	Compliant	Acceptable
Onboarding	non-differentiating			
Rehydration	Easy	Tricky	Easy	Easy
Readability	Least	Moderate	Moderate	Most
Tooling Support	non-differentiating			
Validation	non-differentiating			
Extensibility	Less	Less	Less	Most

See NPT-003 Options Analysis[13] for more discussion.

3.3 Consequences

- Suppliers can leverage existing FHIR tooling, or will likely want to make use of free off-the-shelf libraries if they don't already
- Inbound messages will need to contain all of the information needed to rehydrate a triage to the current state
- Care will need to be taken with extensibility, to ensure new content is delivered in appropriate (compliant) FHIR resources and fields

3.4 Compliance

A manual task is planned to dry-run a triage, examining what we propose the payload would look like and confirming we can extract the required information on either side (i.e., client and server).

13. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1087792387/NPT-003+Options+Analysis

3.5 Notes

The engine determining next steps in a pathways triage can be developed separately behind a well-defined API, with the choice of FHIR Tasks as the infrastructural communication channel acting as an adapter to that core.

3.6 Actions

4 ADR-API-004: Triage API integration with other NHS APIs

- 1. Context
- 2. Decision
  - 2.1. Assumptions
  - 2.2. Drivers
  - 2.3. Criteria
  - 2.4. Options
  - 2.5. Recommendation
  - 2.6. Rationale
- 3. Consequences
- 4. Compliance
- 5. Notes
- 6. Actions

Date	24 Jul 2025
Status	IN PROGRESS
Deciders	Engineering, Architecture, Technical Review and Governance
Significant	Non-functional characteristics
Owners	Gareth Dempsey Ali Rawashdeh
Decision	
Decision waiver	

4.1 Context

The Pathways Triage API is a new digital service intended to facilitate real-time patient triage assessment at national scale. Integrating with this API will remove the need for host system suppliers to implement their own bespoke triage engines.

At present, when handling an emergency call from a member of the public, host systems can make requests to several NHS-hosted APIs/services. This includes:

- APIs/services to determine the patient context, including:
  - Personal Demographics Service[14] - to retrieve a callers demographic information Personal Demographics Service
  - Summary Care Record[15] - to retrieve information such as current medication and allergies
  - GP Connect[16] - to access a callers' GP records
  - Child Protection Information Sharing (CP-IS) service[17] - to retrieve child protection-related data
- Repeat Caller Service[18] - to identify users who have been processed through the service 3 times or more in a 96 hour period.
- Directory of Services[19] - to obtain details of health and care services relevant to the call
- Continuous Quality Improvement[20] - to submit a summary of the triage call at the end of the call for quality monitoring purposes
- Booking and Referral[21] - to send booking and referral information to other NHS service providers
- Post Event Messaging[22] - to send a copy of the referral to GP systems

Both the requests to the new Pathways Triage API from the host systems and the responses created by the API will include some of the data required by these other APIs. This includes the patient's date of birth, gender and the outcome from the triage. It is therefore worth considering whether it is appropriate to move the responsibility for making the requests to some or all of these APIs away from the host systems and into the Pathways Triage API.

4.2 Decision

4.2.1 Assumptions

- Triage is the core function of the Pathways Triage API, and an efficient triage operation is the primary objective of the API
- Not all data required to allow the Triage API to make the calls to these other APIs is necessary for the triage process, and would need to be included in the payload specifically for the purpose of making these additional API calls
- Adding additional data to the payload that is not required for triage will unnecessarily bloat it, and will increase processing time of the request/response within the API, and by the consuming system

14. https://digital.nhs.uk/developer/api-catalogue/personal-demographics-service-fhir
15. https://digital.nhs.uk/services/summary-care-records-scr
16. https://digital.nhs.uk/services/gp-connect
17. https://digital.nhs.uk/services/child-protection-information-sharing-service
18. https://digital.nhs.uk/services/repeat-caller-service
19. https://digital.nhs.uk/developer/api-catalogue/directory-of-services-urgent-and-emergency-care-rest
20. https://digital.nhs.uk/developer/api-catalogue/urgent-and-emergency-care-continuous-quality-improvement-api
21. https://digital.nhs.uk/developer/api-catalogue/booking-and-referral-fhir
22. https://developer.nhs.uk/apis/uec-tech-standards/post_event_messaging.html

4.2.2 Drivers

- Reducing the overall time taken to triage the patient
- Expectation from the host system suppliers that the Triage API would take responsibility for making these API calls with the move to a new centralised API model
- Following good architectural practice in maintaining separation of concerns (e.g. loosely couple services) and aiming for high coherence within a domain.

4.2.3 Criteria

The following criteria was used to assess the change and make a recommendation

- **Consumers impacted:** What consumers of the API would be impacted by this change, only the host system suppliers or also internal consumers of the API (111 Online and S&R)
- **Supplier expectation:** Do host system suppliers need or expect this call to be made from the Triage API. Feedback from user research relating to this can be found here Triage API Responsibilities.pptx[23].
- **Alignment:** Does making this API call from the Triage API align with NHS standards, for example architecture and separation of concerns
- **Complexities:** Does the API have all the data required to make the API request, would it impact latency, etc.
- **Benefit:** Does it yield additional benefit to the NHS to change the current behaviour and make this call from the Triage API

4.2.4 Options

1. **Option 1** - The host systems continue to make **all** calls to the existing NHS APIs listed above, in addition to the calls to the Pathways Triage API.
2. **Option 2** - The Pathways Triage API calls one or more of the NHS APIs. The host system is no longer required to make the call directly to these specific NHS APIs, but will continue to call those APIs not integrated with the Pathways Triage API.
3. **Option 3** - The Pathways Triage API calls **all** of the NHS APIs. The host system is only required to make one call to the Pathways Triage API.

4.2.5 Recommendation

Option 1 is recommended, that the host systems continue to make **all** calls to the existing NHS APIs listed above, in addition to calling the new Pathways Triage API. The rationale for this decision is detailed below.

---

23. https://nhs.sharepoint.com/:p:/r/sites/msteams_9646b6/Shared%20Documents/General/Workstreams/Improving%20how%20we%20Connect%20-%20Triage%20API%20Alpha/Triage%20API%20Responsibilities.pptx?d=w6ea8fac61a2a4bda9af3405df6d460ef&csf=1&web=1&e=yeGX53&nav=eyJzSWQiOjIxNDc0ODI2MTQsImNJZCI6MzkwMTk4MDU4N30

4.2.6 Rationale

	Patient context	Repeat Caller Service[24]	Directory of Services[25]	Continuous Quality Improvement[26]	Booking and Referral[27]	Post Event Messaging[28]
Description	Consuming systems look up patient demographics using PDS. There is also an additional lookup to CPIS for a Child Protection care plan flag. Separate to these, as part of a clinical triage, the clinical call handler will have access to SCR, GP Connect.	Queries and updates to the Repeat Caller Service, helping to route repeat callers to clinicians	Using the triage outcome to look up DoS services appropriate to the patient	The submission of triage audit data to IDT, closing the feedback loop for pathways data	Referral of the patient to the selected service, providing the patient and triage outcome details	A copy of the referral is provided to GP systems
Consumers impacted	Host systems only	Host systems only	Host systems and internal consumers	Host systems and internal consumers	Host systems and internal consumers	Host systems and internal consumers
Supplier expectation	Do not expect this to change	Unknown	Do not expect this to change	Yes, expect this change	Do not expect this to change	Do not expect this to change

24. https://digital.nhs.uk/services/repeat-caller-service
25. https://digital.nhs.uk/developer/api-catalogue/directory-of-services-urgent-and-emergency-care-rest
26. https://digital.nhs.uk/developer/api-catalogue/urgent-and-emergency-care-continuous-quality-improvement-api
27. https://digital.nhs.uk/developer/api-catalogue/booking-and-referral-fhir
28. https://developer.nhs.uk/apis/uec-tech-standards/post_event_messaging.html

Alignment	No, doesn't separate concerns if grouped in a Triage API	Partial. RCS Search could fit in triage, but handling updates wouldn't separate concerns	No, doesn't separate concerns if grouped in a Triage API	Yes This is triage audit data so relevant to a Triage API	No, doesn't separate concerns if grouped in a Triage API	No, doesn't separate concerns if grouped in a Triage API
Complexities	We use v. limited subset of this data during triage. Detailed demographics are not required. Need to handle PII	Need to handle PII	Majority of data required is generated by the host systems and is not required for triage	Majority of the data required is generated by the host systems and is not required for triage	Need to handle PII	Need to handle PII
Benefit	Onboarding benefit for new consumers only	Onboarding benefit for consumers only	Onboarding benefit for consumers only	Yes Improved data completeness, in addition to onboarding benefit.	Onboarding benefit for consumers only	Onboarding benefit for consumers only
Recommendation	Exclude	Exclude (See Notes)	Exclude	Exclude (See Notes)	Exclude	Exclude

4.3 Consequences

- The responsibility for any submissions e.g. UEC CQI API, ITK messaging sits with the host systems
- The Triage API must ensure it is providing sufficient data in any response to the consuming system to allow these calls to be made

4.4 Compliance

- API design guiding principle[29] relating to minimising the information requested from the API consumer

29. https://nhsd-confluence.digital.nhs.uk/spaces/APM/pages/102649419/Designing+your+API#DesigningyourAPI-design-principlesGuidingdesignprinciples

4.5 Notes

1. The inclusion of the call to the CQI API could be included in Triage API in the future if required.
2. Making the call to the Repeat Caller Service, NRL or a future Patient Data Manager (PDM) hosted Single Patient Record could be considered for inclusion when Personalisation is explored

4.6 Actions

5 ADR-API-005: Choice of Triage API PoC development language

Item	Example
Date	12 Aug 2025
Status	`Proposed, Withdrawn, Rejected, Accepted, Deprecated, ..., Superseded by ADR-XXX or Supersedes ADR-XXX`
Deciders	`Tech Radar, Engineering, Architecture, Solution Assurance, Clinical Assurance, Technical Review and Governance, Information Governance, Cyber Security, Live Services Board,` …
Significant	`Structure, Nonfunctional characteristics, Dependencies, Interfaces, Construction techniques,` …
Owners	Afzal Mufti
Decision	`approved, conditional approval, consult with P&P DA, consult with Data Architecture, submit to TRG, rejected`
Decision waiver	`conditions placed on the approval (e.g. approved as a tactical solution, strategic solution to be implemented by <date>, etc.)`

5.1 Context

As part of development of the Triage API technical proof of concept we need to decide on what development language to use.

Learnings from the POC can be taken forwards into Private Beta phase if approved.

As this is a poc the principle of it being 'throw away' should be maintained with 'just enough' development to prove or disprove the assumptions being tested.

The choice of language will impact on what libraries and development approach to take, as well as potentially infrastructure.

Considerations:

- Appropriateness to the solution
- Familiarity of the language / tech stack of the development team
- Familiarity of the language / tech stack within the programme / DUEC / P&P
- Organisational guidance from Engineering Technical Authority
- Risk of additional operational burden (context switching etc) for managing a mixed economy of development languages, tech stack and tooling

5.2 Decision

5.2.1 Assumptions

- This is choice of language for the technical poc. If during the course or at the end of the poc the language choice is not a good fit, then this learning should be taken forwards as part of any beta build.
- NHS England are moving towards consolidation of development languages in the Tech Radar
- Python and TypeScript are preferred for new project development. Java, C# will move to Contain. They will continue to be supported for existing products but no new projects should adopt these languages without engineering board approval

5.2.2 Drivers

- As of August 2025 a proposal for consolidation of development languages has not been formally published but has been circulated in draft format. It has received support from Platforms monthly tech leads meeting so has a likelihood of being formally adopted.
- If this proposal is adopted and the chosen poc language is *not* Python, then there is a risk of misalignment with wider org policy. Additionally, there could be a longer term ask for teams to plan for alignment going forwards.

5.2.3 Criteria

Provide a list of criteria (expected to be a least 5) used to evaluate options. These will typically include several relevant non-functional characteristics of the product/service/capability such as performance, security, operations, reliability, sustainability, costs, technology maturity, team skillset, or strategic direction.

5.2.4 Options

5.2.4.1 C#

- Already in use within Pathways products: PAT, VPAT, Pathways Web, 111 Online but not widely outside of DUEC and nhs.uk
- Familiar to the Pathways BAU team, which may reduce friction at handover

- Expected to be categorised as "Contain" in the proposed Engineering Technical Authority guidance. New projects will require board approval (Engineering, TRG).

5.2.4.2 Python

- FTRS are moving from PHP to Python for their transformation project
- Wide use across NHS England
- Expected to be categorised as "Mainstream" in the Engineering Technical Authority guidance, preferred for new development
- May introduce some friction at BAU handover or need some upskilling from the incumbent team

5.2.4.3 Java

- Has some benefits for Lambda usage (ways to speed up execution)
- Not in wide use within NHS England.
- Expected to be categorised as "Contain" - not recommended for new projects in the Engineering Technical Authority proposal

5.2.5 Recommendation

Use Python

- Decision is reversible at transition to Beta phase
- Aligns with expected org position around consolidating development languages
- Supports use of shared libraries, reusable components and standardised patterns

5.2.6 Rationale

- Aligns with expected Engineering Technical Authority position to be published in due course
- This is a new development and is currently at the Alpha stage. The decision is reversible at the transition to Beta phase if needed.
- Can leverage wider org patterns, learning, tooling, expertise around Python

5.3 Consequences

- Current pathways team will need upskilling to support a python based service if taken forwards to beta and live.

Describe the resulting context, after applying the decision. All the identified consequences should be listed here, not just the positive ones. Any decision comes with many implications. For example, it may introduce a need to make other decisions as an effect of cross-cutting concerns; it may impact structural or operational characteristics of the software, and influence non-functional requirements; as a result, some things may become easier or more difficult to do because of this change. What are the trade-offs?

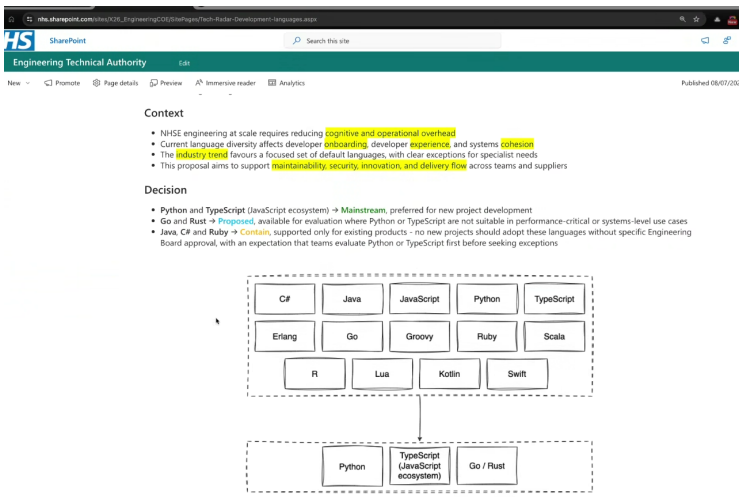
What are the conditions under which this decision no longer applies or becomes irrelevant?

5.4 Compliance

Establish how the success is going to be measured. Once implemented, the effect might lend itself to be measured, therefore if appropriate a set of criteria for success could be established. Compliance checks of the decision can be manual or automated using a fitness function. If it is the latter this section can then specify how that fitness function would be implemented and whether there are any other changes to the codebase needed to measure this decision for compliance.

5.5 Notes

Draft proposal for language consolidation:



Consequences

- Simplifies hiring, onboarding, and upskilling across internal teams and suppliers
  - Enables smoother movement across teams, improving delivery efficiency (people become effective more quickly in new teams), supporting retention (lower friction to explore new challenges), and strengthening knowledge sharing across the organisation
  - Provides clear guidance for suppliers on preferred technologies, helping align delivery expectations, reduce onboarding time, and ensure better integration with internal teams and platforms
- Encourages shared libraries, reusable components, and standardised SDLC practices
- Reduces platform support overhead for internal tooling, security scanning, and CI/CD pipelines
- Provides clearer guidance for new project setup and delivery partner alignment
- Supports the gradual deprecation of legacy technologies without imposing rigid migration deadlines
- Improves security posture through standardisation across the SDLC
  - Easier vulnerability management, static analysis, dependency tracking, and secure-by-design scaffolding
- Supports strategic investment
  - We invest in a smaller number of platforms, tools, and capabilities
  - Enables better long-term returns on investment in areas like training, automation, and support
- Reinforces meaningful innovation
  - E.g. are we trying to invent new network protocols or solve novel healthcare problems?
  - This proposal helps focus innovation effort on what matters most - improving patient outcomes and healthcare delivery
- Clarifies this is not a judgment on language quality
  - Java and C# are mature, capable languages
  - **This is a business decision based on consolidation, current usage (with Python and TypeScript being the most widely adopted), business fit, and industry direction.**

Key considerations

- Can we recruit at scale in this language? Comments:

> *"Most if not all of the candidates I interviewed during recruitment campaigns for engineering roles had experience with either Python or TypeScript, or both."*
> *— NHSE Recruitment Manager*

5.6 Actions

- [ ] name, date by, action
- [ ] name, date by, action

6 ADR-API-006: Triage API Architecture

Date	24 Sept 2025
Status	Draft
Deciders	NHS England, DUEC SMT, Architecture & Engineering
Significant Owners	SaET Team
Decision	Adopt a stateless triage API hosted on AWS, exposed via NHS England APIM

6.1 1. Context

The triage API must support critical 111 and 999 services, returning a question-and-answer flow that leads to safe care advice and a triage outcome.

The API must:

- Be **stateless**, with consumers managing state as agreed in ADR-API-003: Triage API communication format options (see page 27)
- Operate through **NHS England API Management (APIM)** for authentication, authorisation, throttling, and centralised monitoring.
- Support **correlation of requests** using a UUID for auditability and investigations.
- Scale to meet national demand while remaining secure and compliant.

6.2 2. Business Drivers

- Enable national consistency of triage logic and outcomes.
- Reduce supplier-specific variation and repeated downstream triage.
- Support resilience and horizontal scalability to meet national demand.
- Maintain clinical reproducibility and traceability of every outcome.
- Align with NHS API Management, security, and information-governance standards.
- Facilitate future interoperability (e.g. FHIR / SNOMED CT alignment).

6.3 3. Decision

6.3.1 3.1 Options

Option	Description	Pros (For)	Cons (Against)	Recommendation / Rationale
Option 1 – Monolithic, Stateful API	API maintains conversation state server-side and manages session persistence for consumers.	• Simpler client integration • Easier to debug user journeys • Potentially fewer supplier-side responsibilities	• Harder to scale horizontally • Higher infrastructure and maintenance costs- Increased complexity and fault domains • Contradicts NHS principle of “stateless, scalable APIs”	**Rejected** – scalability, resilience, and separation of responsibilities make this unsuitable for national-scale use.
Option 2 – Stateless API with Client-Managed State	API only processes the current request and returns the next question or outcome; client maintains state.	• Highly scalable and horizontally distributable • Easier fault isolation and load balancing • Works cleanly with NHS APIM • Aligns with modern API patterns	• Puts more responsibility on suppliers • Additional responsibility on the onboarding and conformance approach, requiring strict technical assurance to be confident in the deployment and responsible usage in live environment. • Requires clear documentation and error handling standards	**Selected** – offers the best balance of scalability, security, maintainability, and alignment with NHS architectural principles.

Option	Description	Pros (For)	Cons (Against)	Recommendation / Rationale
Option 3 – Hybrid (Stateless API with Server-Side Session Store)	API remains stateless for logic but stores temporary session context on the server.	• Reduces supplier complexity • Easier debugging of sessions	• Adds state management overhead • Complicates scaling and resilience • Creates potential for partial failures and data consistency issues	**Rejected** – adds unnecessary complexity while undermining the benefits of a truly stateless architecture.

6.3.2 3.2 Recommendation

Adopt Option 2 – Stateless API with client-managed state.

- Deploy into AWS VPC across multiple AZs (see ADR-003 and ADR-014).
- Enforce API ingress via NHSE APIM (see ADR-013).
- Use UUID correlation ID for traceability.
- Publish clear integration guidance for suppliers on state handling.
- Ensure payload design aligns with FHIR Task decision (ADR-004).
- Maintain architectural separation as defined in ADR-008.

6.4 4. Consequences

- Positive: Highly scalable, simple fault isolation, compliant with NHS standards.
- Negative: Requires supplier maturity in managing state.

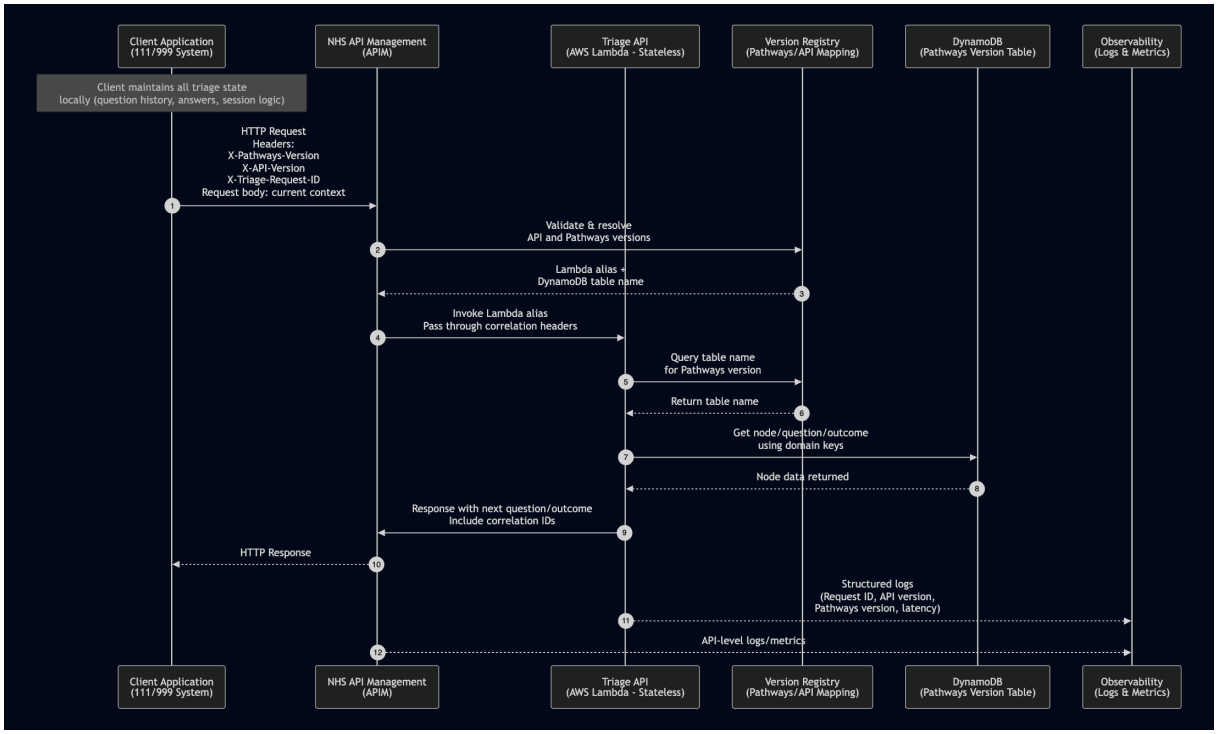
6.5 5. Compliance

- Meets NHS Cloud First and API First principles.
- Aligns with DSPT, NHS Cyber Security, and ISO 27001.
- Enforces APIM as mandated by NHSE.

6.6 6. Actions

- Define API contract for question/answer flow.
- Publish supplier-facing guidance for state management.
- Implement UUID correlation logging.
- Deploy stateless API workloads into AWS private subnets behind APIM.

6.6.1 Stateless flow diagram



7 ADR-API-007: Triage API Versioning

Date	24 Sept 2025
Status	Draft
Deciders	NHS England, DUEC SMT, Architecture & Engineering
Significant Owners	SaET Team
Decision	Adopt semantic versioning with explicit deprecation timelines, enforced through APIM

7.1 1. Context

The Triage API will be consumed by multiple suppliers (111 and 999 systems) via NHS England's **API Management (APIM)** platform.
Each supplier may adopt updates at different times; therefore, version control is essential to ensure **stability, clinical reproducibility, and backward compatibility** across the ecosystem.

The API's versioning strategy must:

- Allow suppliers to continue using stable versions without disruption.
- Support clear governance and traceability for clinical audit.
- Comply with **NHS England APIM policies**, which favour header-based versioning for internal routing and governance consistency.

Because triage logic and outcomes are clinically sensitive, **every API version must remain traceable to a defined Pathways content version** and undergo formal governance review prior to deprecation or retirement.

The patient triage Api will follow the versioning strategy for APIs outlined by APIM here: Version control[30] which follows https://semver.org/ distinction of Major and Minor changes. You can see some clear examples of a Major and Minor change from APIM here[31].

We have two reasons for wanting to publish an API versioning and deprecation policy:

1. Different host systems maybe on different versions of the system. We do not want to force host systems onto versions they have not yet trained on.
2. Early adopters will require a way of to target the latest version without affecting other host systems.

30. https://nhsd-confluence.digital.nhs.uk/spaces/APM/pages/144742522/Version+control
31. https://nhsd-confluence.digital.nhs.uk/spaces/APM/pages/457622435/API+versioning+analysis#APIversioninganalysis-Whyshouldweintroduceversioning?

There is no current NHS England API Versioning strategy or policy. There is however definitions of deprecation and retirement[32]

7.2 2. Business Drivers

- **Supplier assurance:** Enable predictable adoption of new versions.
- **Clinical safety:** Avoid unplanned behaviour changes across suppliers.
- **Governance:** Ensure NHS oversight of deprecation and upgrade timelines.
- **Compliance:** Align with NHS architectural principles on API lifecycle management.

7.3 3. Decision

7.3.1 3.1 Options

Option	Description	Advantages	Disadvantages
Option 1: Query parameter versioning	Version information is provided as a query string parameter (e.g. `/triage?version=1`). Routing decisions and version control logic are handled by the application itself or a custom APIM policy reading the query parameter.	• Easy to implement in basic APIs. • Requires minimal supplier change for testing. • Versions can be changed dynamically without altering the endpoint path.	• **Not aligned with NHS APIM design standards**, which discourage routing based on query parameters. • Query parameters are often stripped or cached incorrectly by proxies. • Difficult to log and audit at scale since version is buried in URL parameters. • Creates ambiguity when combined with other query arguments.

32. https://digital.nhs.uk/services/care-identity-service/releases-and-planned-maintenance/service-deprecation-and-retirement#retired-services

Option	Description	Advantages	Disadvantages
Option 2: Semantic versioning with strict deprecation policy	Major, minor, and patch versions are encoded in the URI path (e.g. `/v1/triage`, `/v2/triage`). Each major version represents a breaking change, and older versions are retained until decommissioned through a published deprecation cycle.	• Clear separation of major versions, easily visible to integrators.• Well understood by most API developers. • Simplifies documentation and onboarding because each version is a discrete endpoint. • Enables safe migration by maintaining legacy routes until suppliers transition.	• Requires **duplicate APIM routes and policies** for each version, increasing operational overhead. • Harder to apply consistent security and throttling rules across all versions. • Changes to URI paths can disrupt cached references or supplier integrations. • Not the preferred approach in NHS APIM due to maintenance and governance complexity.
Option 3: Header-based Versioning (selected)	The API version is passed through the accept header and the parameter, e.g. **application/fhir+json; version=2**. The APIM layer reads this header to route the request to the appropriate backend deployment or configuration. The endpoint path remains constant, ensuring stability and clean URLs.	• **Fully compatible with NHS APIM policies** which support header-based version routing. • Keeps URIs stable and supplier-friendly while enabling concurrent version operation. • Simplifies governance and documentation by maintaining a single logical endpoint. • Enables version negotiation and default version fallbacks in APIM. • Reduces duplication of routes and configuration drift.	• Requires suppliers to always send the correct version header (must be enforced and validated). • Slightly more complex to debug manually since versioning info is in headers, not URL. • Requires clear documentation and validation tooling to avoid missing or invalid version headers.

Option	Description	Advantages	Disadvantages
Option 4: Self managed versioning	The client supplies a client Id which is used to lookup the clients configuration record. The record will contain the API version as well as the clients clinical version. This is then used to route to the correct Api and clinical version. Architecture 7.1.2.2.9.1 - Triage self managed versioning architecture[33]	• Client has no configuration once they are set up • A single database lookup returns both the API version and the client data version. • Providers cannot supply invalid or mismatched API and client data version combinations. • Versioning is centrally controlled and managed. • Offers the option for further extensibility in future. • Less reliant on the APIM routing	• Configuration must be set up ahead of any migration or upgrade to a newer version. • Providers must provide their API and client data version details before they can complete a migration. • Additional workload for the triage team to update provider records when moving them to newer versions. • More processing required to get the clients configuration each time they access the Api, although caching may mitigate this.

7.3.2 3.2 Recommendation

Adopt Option 3 – Header-based Versioning

This approach:

- Aligns with **NHS APIM's version management and routing** model.
- Keeps endpoint URIs stable and clean.
- Simplifies supplier onboarding through consistent API behaviour.
- Enables multiple versions to run concurrently within APIM without duplicating endpoints.
- Follow APIMs guidance for versioning
- Publish an API versioning and Deprecation Policy below as part of our OAS spec
- Include the policy, or a link, in the connection agreement connecting parties and data partners sign as part of onboarding to the API

Each API version will be linked to one or more **compatible Pathways content versions**, ensuring clinical safety and reproducibility.

The Triage API will adopt **header-based versioning**, using a dedicated request header to specify the required API version.

33. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1233924356/7.1.2.2.9.1+-+Triage+self+managed+versioning+architecture

7.3.2.1 Implementation pattern

Clients will include the following header in all API requests:

```
application/fhir+json; version=<major version>
```

For example:

```
application/fhir+json; version=2
```

If no version is specified, the API will default to the **latest supported version** (configurable via APIM policy).

7.4 4. Consequences

- Clients must implement and maintain the version header in all requests.
- API Gateway policies must validate and route based on version headers.
- Governance and documentation processes must track header-based version lifecycles.
- Deprecation notices will need clear publication and supplier engagement.

7.5 5. Compliance

- Aligns with NHS API standards and API First principle.
- Meets governance expectations for transparency and lifecycle management.

7.6 6. Actions

- Define semantic versioning policy and publish guidance.
- Configure APIM to route requests by version.
- Establish governance process for approving breaking changes.
- Communicate deprecation timelines to suppliers in advance.

8 ADR-API-008: Pathways Content Versioning

Date	25 Sept 2025
Status	Draft
Deciders	NHS England, DUEC SMT, Architecture & Engineering
Significant Owners	SaET Team
Decision	Adopt immutable versioning for Pathways content, with governance-controlled release and deprecation policy

8.1 1. Context

The triage API consumes **Pathways clinical content**, which evolves regularly to reflect clinical safety updates and policy changes. It is essential that suppliers can reproduce triage outcomes reliably and that clinical governance has oversight of changes.

Unlike API functionality, Pathways content carries **direct patient safety implications**. Content changes must therefore be versioned immutably and released under controlled governance.

8.2 2. Business Drivers

- Ensure clinical reproducibility and auditability of all triage outcomes.
- Enable version control and rollback of Pathways content.
- Provide stable release interfaces for supplier integrations.
- Support governance, assurance, and clinical safety processes.
- Maintain compatibility between content releases and API versions.
- Align with the Triage Outcome Model (TOM) and future SNOMED CT mapping work.

8.3 3. Decision

8.3.1 3.1 Options

Option	Description	Advantages	Disadvantages
Option 1: Overwrite existing content in place	Replace existing triage content in the database with the new release.	• Simple implementation. • Minimal storage growth.	• No rollback path. • Breaks reproducibility and auditability. • Not compliant with clinical governance.
Option 2: Immutable versioned storage (selected)	Each release is stored as a separate, immutable version. Metadata records track version details and compatibility.	• Fully traceable and auditable. • Enables rollback and comparison testing. • Supports concurrent versions for transition. • Aligns with governance standards.	• Increased storage requirements. • Requires version management logic in ingestion and API.
Option 3: Delta or patch-based updates	Store only the differences between releases (e.g. incremental changes).	• Reduces data storage and load times. • Easier differential testing.	• Increases complexity. • Difficult to validate integrity. • Error-prone for rollback or audit reconstruction.

8.3.2 3.2 Recommendation

Adopt Option 2 – Immutable versioning with governance-controlled lifecycle.

Pathways content will be **versioned, immutable, and independently traceable** from API logic.
Each content release will have a unique **semantic version identifier** (e.g. `2025.2.1`) and will be **governed through the clinical and technical release pipeline**.

All versions will be stored in a dedicated, versioned collection within the Pathways DynamoDB tables. The ingestion process (see ADR-017) will load new versions, validate schema integrity, and tag them with metadata including:

- **Version number**
- **Release date**
- **Clinical governance reference**
- **Compatibility range of API versions**

- **Checksum / hash for content validation**
- **Status** (Active, Deprecated, Archived)

Older versions will remain accessible in a read-only state for governance and audit until formally retired.

8.4 4. Consequences

- Positive: Ensures reproducibility, enables clinical audit, and increases supplier confidence.
- Negative: Increased operational overhead to manage content versions.

8.5 5. Compliance

- Aligns with NHS clinical safety case requirements.
- Supports DSPT and ISO 13485 principles for traceability of clinical systems.

8.6 6. Actions

- Define versioning scheme for Pathways content.
- Implement governance process for approval of content releases.
- Enhance API logging to include content version identifiers.
- Publish supplier-facing release notes for each content update.

9 ADR-API-009: [WIP]Choice of Triage API Authentication Pattern

Item	Example
Date	08 Sept 2025
Status	`Proposed`
Deciders	`Tech Radar, Engineering, Architecture, Solution Assurance, Clinical Assurance, Technical Review and Governance, Information Governance, Cyber Security, Live Services Board,` ...
Significant	`Structure, Nonfunctional characteristics, Dependencies, Interfaces, Construction techniques,` ...
Owners	Afzal Mufti
Decision	`approved, conditional approval, consult with P&P DA, consult with Data Architecture, submit to TRG, rejected`
Decision waiver	`conditions placed on the approval (e.g. approved as a tactical solution, strategic solution to be implemented by <date>, etc.)`

9.1 WIP

Write up of the below into an ADR:

This API uses the following security pattern(s):

- application-restricted access - using API key authentication (as per API consumer security - Application restricted- API key authentication[34])
  - This pattern is intended for Alpha phase for speed and ease of implementation. Alpha phase will cover a small subset of clinical pathways.
- application-restricted access - using signed JWT authentication (as per API consumer security - Application restricted API- signed JWT authentication[35])

34. https://nhsd-confluence.digital.nhs.uk/display/APM/API+consumer+security#APIconsumersecurity-Application-restrictedAPI-APIkeyauthentication

    - This pattern is intended once we proceed to Private Beta.
- ~~user-restricted access - using NHS Identity for end user authentication (end user is a healthcare professional) (as per API consumer security- User-restricted-NHS CIS2[36])~~
- ~~user-restricted access - using NHS Login for end user authentication (end user is a citizen) (as per API consumer security- User-restricted-NHS login[37])~~

This pattern is appropriate because:

- The API is read-only. There is no facility to add, update or delete data.
- PID is not required for Triage
- The API is stateless. The host system is responsible for managing the Triage Journey state.
- A call to the api is to determine the next best activity - ask another question, or triage outcome. There is no single call that can be made to elicit large scale export of the algorithm. It would need continuous cycling of inputs to spider the algorithm. Rate limits and WAF will be in place to mitigate this including the app-restricted protections.
- Telephony call handlers do access sensitive patient data within the host system and it is the host system responsibility to authenticate users for this.

The above elements taken together mean this

- For self triage route - if using 111OL via the NHS App then the login is passed to 111 Online to pre-fill some elements if there is a referral.
- For telephony, call handlers are not normally employed by NHS but by the healthcare provider meaning they typically do not have nhs.net[38] accounts. Smartcard usage is mixed depending on provider. Rollout of NHS Identity for telephony providers would require significant resource and management. This could impact uptake of the API.

9.1.1 Context

As part of development of the Triage API technical proof of concept we need to decide on what development language to use.

Learnings from the POC can be taken forwards into Private Beta phase if approved.

As this is a poc the principle of it being 'throw away' should be maintained with 'just enough' development to prove or disprove the assumptions being tested.

---

35. https://nhsd-confluence.digital.nhs.uk/display/APM/API+consumer+security#APIconsumersecurity-Application-restrictedAPI-signedJWTauthentication
36. https://nhsd-confluence.digital.nhs.uk/display/APM/API+consumer+security#APIconsumersecurity-User-restricted-NHSCIS2
37. https://nhsd-confluence.digital.nhs.uk/display/APM/API+consumer+security#APIconsumersecurity-User-restricted-NHSlogin
38. http://nhs.net

The choice of language will impact on what libraries and development approach to take, as well as potentially infrastructure.

Considerations:

- Appropriateness to the solution
- Familiarity of the language / tech stack of the development team
- Familiarity of the language / tech stack within the programme / DUEC / P&P
- Organisational guidance from Engineering Technical Authority
- Risk of additional operational burden (context switching etc) for managing a mixed economy of development languages, tech stack and tooling

9.1.2 Decision

9.1.2.1 Assumptions

- This is choice of language for the technical poc. If during the course or at the end of the poc the language choice is not a good fit, then this learning should be taken forwards as part of any beta build.
- NHS England are moving towards consolidation of development languages in the Tech Radar
- Python and TypeScript are preferred for new project development. Java, C# will move to Contain. They will continue to be supported for existing products but no new projects should adopt these languages without engineering board approval

9.1.2.2 Drivers

- As of August 2025 a proposal for consolidation of development languages has not been formally published but has been circulated in draft format. It has received support from Platforms monthly tech leads meeting so has a likelihood of being formally adopted.
- If this proposal is adopted and the chosen poc language is *not* Python, then there is a risk of misalignment with wider org policy. Additionally, there could be a longer term ask for teams to plan for alignment going forwards.

9.1.2.3 Criteria

Provide a list of criteria (expected to be a least 5) used to evaluate options. These will typically include several relevant non-functional characteristics of the product/service/capability such as performance, security, operations, reliability, sustainability, costs, technology maturity, team skillset, or strategic direction.

9.1.2.4 Options

9.1.2.4.1 C#

- Already in use within Pathways products: PAT, VPAT, Pathways Web, 111 Online but not widely outside of DUEC and nhs.uk
- Familiar to the Pathways BAU team, which may reduce friction at handover
- Expected to be categorised as "Contain" in the proposed Engineering Technical Authority guidance. New projects will require board approval (Engineering, TRG).

9.1.2.4.2 Python

- FTRS are moving from PHP to Python for their transformation project
- Wide use across NHS England
- Expected to be categorised as "Mainstream" in the Engineering Technical Authority guidance, preferred for new development
- May introduce some friction at BAU handover or need some upskilling from the incumbent team

9.1.2.4.3 Java

- Has some benefits for Lambda usage (ways to speed up execution)
- Not in wide use within NHS England.
- Expected to be categorised as "Contain" - not recommended for new projects in the Engineering Technical Authority proposal

9.1.2.5 Recommendation

Use Python

- Decision is reversible at transition to Beta phase
- Aligns with expected org position around consolidating development languages
- Supports use of shared libraries, reusable components and standardised patterns

9.1.2.6 Rationale

- Aligns with expected Engineering Technical Authority position to be published in due course
- This is a new development and is currently at the Alpha stage. The decision is reversible at the transition to Beta phase if needed.
- Can leverage wider org patterns, learning, tooling, expertise around Python

9.1.3 Consequences

- Current pathways team will need upskilling to support a python based service if taken forwards to beta and live.

Describe the resulting context, after applying the decision. All the identified consequences should be listed here, not just the positive ones. Any decision comes with many implications. For example, it may introduce a need to make other decisions as an effect of cross-cutting concerns; it may impact structural or operational characteristics of the software, and influence non-functional requirements; as a result, some things may become easier or more difficult to do because of this change. What are the trade-offs?

What are the conditions under which this decision no longer applies or becomes irrelevant?

9.1.4 Compliance

Establish how the success is going to be measured. Once implemented, the effect might lend itself to be measured, therefore if appropriate a set of criteria for success could be established. Compliance checks of the decision can be manual or automated using a fitness function. If it is the latter this section can then specify how that fitness function would be implemented and whether there are any other changes to the codebase needed to measure this decision for compliance.

9.1.5 Notes

Draft proposal for language consolidation:

9.1.6 Actions

- [ ] name, date by, action
- [ ] name, date by, action

10 ADR-API-010: AWS Infrastructure and API Security

Date	24 Sept 2025
Status	Draft
Deciders	NHS England, DUEC SMT, Architecture & Engineering
Significant Owners	SaET Team
Decision	Deploy the triage API into a private, multi-AZ AWS VPC with APIM ingress, layered security controls, and NHS-aligned governance

10.1 1. Context

The triage API underpins 111 and 999 services and must operate securely, reliably, and at scale. Hosting must meet NHS security and compliance requirements, including DSPT, NHS Cyber Security, and ISO 27001.

This ADR builds upon and must be read in conjunction with:

- **ADR-API-002:** Which establishes the hosting decision and confirms AWS as the target platform.
- **ADR-API-006:** Which defines the stateless API architecture and correlation model.
- **ADR-API-009:** Which sets the authentication and authorisation pattern for the API.

The infrastructure must:

- Deliver **multi-AZ resilience** and automatic scaling.
- Defence-in-depth security architecture (identity, network, encryption).
- Support **secure ingress and identity-based access** via NHS APIM.
- Provide integrated **monitoring, logging, and incident visibility** for both engineering and governance purposes.
- Isolation between environments (development, staging, production).
- Continuous monitoring and compliance validation.
- Compatibility with NHS APIM and enterprise security tooling.

10.2 2. Business Drivers

- Provide a secure and scalable hosting environment for the national Triage API.
- Ensure resilience and business continuity aligned with clinical safety requirements.
- Comply with NHS England cyber and information governance policies.

- Protect sensitive triage and audit data from unauthorised access or loss.
- Enable efficient operation and observability using AWS-native services.
- Ensure alignment with DSPT, ISO 27001, and NCSC cloud security principles.

10.3 3. Decision

10.3.1 3.1 Options

Option	Description	Advantages	Disadvantages
Option 1: Single-AZ deployment	Host API and database within one Availability Zone.	• Lower cost. • Simpler to manage initially.	• Single point of failure. • Not clinically safe. • Fails NHS service continuity expectations.
Option 2: Multi-AZ, private-by-default (selected)	Deploy application and data tiers across at least two AWS Availability Zones within a private VPC.	• High resilience and fault tolerance. • Meets DR and DSPT compliance. • Defence-in-depth via isolation and encryption. • Seamless scaling and failover.	• Higher operational complexity and cost. • Requires careful IAM and network governance.
Option 3: Hybrid (on-prem + AWS)	Split workload between AWS and NHS data centres.	• Reuses existing infrastructure investments • Easier access control for legacy systems.	• Complex to manage and integrate. • Inconsistent latency and security posture. • Not aligned with cloud-first strategy.

10.3.2 3.2 Recommendation

Option 2 – Multi-AZ, private-by-default deployment has been selected.
This approach meets NHS England's resilience, security, and compliance requirements while enabling scalability and observability.
All environments (Development, Staging, Production) will use the same baseline architecture via Infrastructure as Code (IaC).

The API will remain **private**, only exposed externally through **NHS APIM**, which manages authentication, authorisation, and throttling.

[Placeholder – Diagram: AWS Infrastructure]

Layer	Component(s)	Security / Operational Controls
Network Layer	Private VPC with public and private subnets across ≥2 Availability Zones.	Network segmentation, NAT gateways for outbound-only internet access, AWS WAF for inbound protection.
Application Layer	AWS Lambda or ECS Fargate containers hosting stateless API services.	Runs within private subnets; communication restricted via security groups and VPC endpoints.
API Layer	AWS API Gateway (private integration) behind NHS APIM.	Mutual TLS, request signing, input validation, rate limiting, header-based version routing.
Data Layer	AWS DynamoDB with point-in-time recovery and encryption at rest.	Server-side encryption using AWS KMS; IAM-based access; audit logging enabled.
Identity & Access Layer	AWS IAM roles and policies; AWS SSO for administration.	Role-based least privilege; separation of duties; multi-factor authentication (MFA).
Observability & Compliance Layer	CloudWatch, CloudTrail, AWS Config, Security Hub, Splunk/Odin.	Centralised monitoring, alerting, compliance dashboards, and security event integration.

10.3.2.1 Identity and Access Management

- Role-based least privilege enforced using AWS IAM.
- MFA enforced for all administrative access.
- No hardcoded credentials; secrets managed via AWS Secrets Manager.
- Automated role reviews and key rotation.

10.3.2.2 Network and Perimeter

- Private VPC; no direct internet access to compute resources.
- Public ingress limited to APIM and WAF.
- Security groups enforce least privilege between layers.
- AWS Shield and WAF protect against DDoS and injection attacks.

10.3.2.3 **Data Protection**

- Data encrypted at rest (AWS KMS) and in transit (TLS 1.2+).
- Pathways content, logs, and metadata stored in DynamoDB with point-in-time recovery.
- Logs shipped to central SIEM (Splunk/Odin) for retention and audit.

10.3.2.4 **Monitoring and Logging**

- CloudTrail for API-level activity tracking.
- CloudWatch metrics and alarms integrated with NHSE observability stack.
- Automated compliance scanning (AWS Config, Security Hub).
- SIEM integration for incident response.

10.3.2.5 **Backup and Recovery**

- Daily DynamoDB backups and point-in-time recovery (PITR).
- Configuration state managed in version-controlled IaC repositories.
- Annual DR tests in coordination with ADR-API-018.

10.4 4. Consequences

- Positive: Secure, resilient, compliant, aligned with NHS strategy.
- Negative: Requires more complex operational management.

10.5 5. Compliance

- Meets DSPT, NHS Cyber Security, and ISO 27001 requirements.
- Enforces NHS “cloud-first” and “secure by design” principles.
- Provides audit trail for governance and incident response.

10.6 6. Actions

- Implement VPC, subnets, and IAM roles using Infrastructure as Code.
- Configure APIM to route securely into AWS private API Gateway.
- Enable CloudTrail, Config, and Security Hub across all accounts.
- Document security posture for NHS governance boards.
- Document DR, backup, and recovery testing schedules.

11 ADR-API-011: Database Choice

Date	24 Sept 2025
Status	Draft
Deciders	NHS England, DUEC SMT, Architecture & Engineering
Significant Owners	SaET Team
Decision	Adopt AWS DynamoDB as the primary datastore, with audit of alternatives documented

11.1 1. Context

The Triage API must persist and serve **Pathways triage content**, including question flows, answer sets, triage outcomes, and metadata.
The system must provide **fast, predictable, and highly available read performance**, since latency directly affects user experience during clinical triage.
Data must be versioned, immutable, and easily auditable (see ADR-012).

Historically, Pathways data has been stored in relational form; however, the content structure is hierarchical and better suited to a document-oriented model.
Given the API's requirement for resilience, horizontal scalability, and tight integration with AWS infrastructure, a cloud-native database is required.

This ADR builds upon:

- **ADR-003:** Hosting strategy (AWS environment).
- **ADR-010:** Stateless API design and UUID correlation requirements.
- **ADR-014:** Infrastructure and security decisions (network, IAM, encryption).

The workload characteristics:

- Highly scalable and read-intensive.
- Predictable access patterns with simple key-value queries.
- No requirement for complex relational joins or multi-table transactions.

11.2 2. Business Drivers

- Guarantee sub-250 ms read latency under national load.
- Support flexible schema evolution as the triage model evolves.
- Allow efficient versioning and rollback of Pathways content.
- Provide audit and immutability for clinical safety and governance.

- Integrate seamlessly with AWS hosting and NHS APIM architecture.
- Deliver resilience and performance with minimal operational overhead.

11.3 3. Decision

11.3.1 3.1 Options

Option	Description	Advantages	Disadvantages
Option 1: AWS DynamoDB (selected)	Fully managed NoSQL database integrated with AWS. Supports document and key-value storage, automatic scaling, and global secondary indexes.	• Highly available and horizontally scalable. • Sub-millisecond read/ write latency. • Native encryption, backup, and IAM integration. • Ideal for stateless, event-driven workloads. • Schema flexibility supports Pathways evolution. • Point-in-time recovery and cross-AZ durability.	• Limited ad-hoc query capability compared with SQL. • Requires careful partition-key design to optimise throughput. • Complex joins must be handled in application logic.
Option 2: PostgreSQL / Amazon RDS	Traditional relational database hosted on AWS RDS.	• Mature, familiar technology. • Strong transactional integrity and SQL capability. • Easier analytical querying.	• Fixed schema limits flexibility. • Requires active performance tuning and scaling. • Slower under hierarchical content workloads. • Higher maintenance burden.
Option 3: Neo4j or graph database	Graph-oriented model suited to probabilistic or relational inference.	• Ideal for exploring probabilistic relationships or pathway graphs. • Could complement Bayesian triage logic in future.	• Over-engineered for deterministic API phase. • Expensive to host and integrate. • Would add unnecessary complexity at MVP stage.

11.3.2 3.2 Recommendation

Option 1 – AWS DynamoDB has been selected.
It satisfies NHS scalability and resilience requirements while allowing flexible schema evolution as the triage logic matures.
Its native AWS integration simplifies operations, monitoring, and compliance, and aligns with the multi-AZ, private-by-default architecture described in ADR-014.

- Use DynamoDB tables to store UUID correlation data, transaction metadata, and Pathways content version references.
- Enable point-in-time recovery for resilience.
- Integrate with IAM, KMS, and CloudWatch as defined in ADR-014.
- Document fallback and migration strategies should future relational workloads emerge.

11.4 4. Consequences

- Reduced operational overhead due to fully managed service.
- Query flexibility limited; analytical queries require export to S3/Athena.
- Application layer responsible for handling joins and composite queries.
- Enables faster ingestion and lower latency for runtime triage.
- Supports future integration with analytical or probabilistic engines (e.g. Neo4j) without schema redesign.

11.5 5. Compliance

- Aligns with **ADR-010 (API Architecture)** and **ADR-014 (AWS Infrastructure)**.
- Supports clinical safety audit and version control under **ADR-012 (Content Versioning)**.
- Fully compliant with NHS DSPT, NCSC, and AWS security standards (encryption, IAM, PITR).
- Read/write operations logged to CloudTrail and SIEM (Splunk/Odin) per **ADR-018 (Monitoring)**.

11.6 6. Actions

- Implement DynamoDB schema as described above with IaC templates.
- Configure backups, PITR, and encryption using AWS KMS.
- Define partition-key strategy to optimise access patterns.
- Build ingestion and validation logic (see ADR-017).
- Establish export mechanism to S3/Athena for audit and analysis.

12 ADR-API-012: Triage API Data Model

Date	24 Sept 2025
Status	Draft
Deciders	Architecture & Engineering
Significant Owners	SaET Team
Decision	Define a normalised triage data model capturing inputs, outcomes, UUID correlation, and Pathways content version

12.1 1. Context

The Triage API stores NHS Pathways clinical triage content that drives question/answer flows and outcomes. The data model must deliver deterministic traversal, national-scale performance, strict version control, auditability, and rapid rollback, while aligning with a stateless API and NHS governance. Each item in the content hierarchy has both **clinical meaning** and **technical relationships** that need to be represented efficiently, traceably, and in a way that supports **stateless retrieval**.

An Alpha implementation validated the triage engine's **hexagonal architecture**, a CSV → domain → persistence pipeline, and DynamoDB as the persistence layer. We will require **hard isolation between versions**, the ability to **introduce future SNOMED CT mapping changes** without impacting historical behaviour, and the ability to **route specific Lambda/API versions to specific content versions**.

The model must:

- Deliver fast question/answer traversal at national scale
- Avoid runtime joins or graph queries
- Remain extensible to future TOM evolution
- Follow **domain-driven design**, not a direct CSV schema or relational mapping
- Remain **lightweight** to avoid introducing server-side state management responsibilities
- Include version-scoped isolation of Pathways content
- Ensure clinically safe, reproducible behaviour for each released content version
- Support **multiple concurrent versions** with **strict boundaries**
- Enable rapid **rollback** and future **schema extensions** (e.g. SNOMED CT) without risk to prior versions
- Keep end-to-end latency within targets under peak load
- Ensure full audit traceability as well as provide unambiguous audit linkage between a triage outcome and the exact content set used

This ADR builds upon:

- **ADR-010:** Defines the stateless API design.
- **ADR-012:** Defines versioning and delivery of Pathways content that must be referenced in the model.
- **ADR-014:** Establishes infrastructure and security patterns for storing data.
- **ADR-015:** Selects AWS DynamoDB as the underlying datastore.

12.2 2. Business Drivers

- Ensure clinical content is structured and retrievable at high speed.
- Provide a consistent, version-controlled schema supporting immutable Pathways releases.
- Enable correlation and reproducibility across triage sessions.
- Support future analytics and interoperability (FHIR, SNOMED CT).
- Maintain compliance with NHS information-governance and DSPT requirements.

12.3 3. Decision

12.3.1 3.1 Options

Option	Description	Pros	Cons
Option 1: Relational Database (PostgreSQL / RDS)	Model triage content in a normalised relational schema (tables for questions, answers, dispositions, relationships). Use joins to navigate pathways flow.	Well-understood model, strong reporting/ querying, transactional consistency, mature tooling.	High latency due to join depth, complex schema evolution as content changes, poor horizontal scaling, more operational overhead, rollbacks require data mutation not version switching.
Option 2: Graph Database (Neo4j / Neptune)	Store triage nodes as vertices and transitions as edges, allowing graph traversal for question → answer → next-node resolution.	Natural fit for graph-like decision flow, flexible traversal logic, good for path analysis and knowledge graph extensions.	Unnecessary complexity — triage engine already encodes traversal, higher latency risk, increased ops overhead, DLL update complexity, harder version isolation, rollback not clean without duplicating graph per version.
Option 3: Multiple DynamoDB Tables (Table-per-entity or per version)	Use separate tables for questions, variables, pathways versions or entity groups.	Clear conceptual boundaries, easy per-entity queries, versioning physically separated.	Increased cost and complexity, cross-table joins not supported, multiple round trips, brittle version toggle logic, difficult to scale operationally, harder to enforce DDD boundaries.

Option	Description	Pros	Cons
Option 4: S3 JSON access (Direct file reads)	Store content as JSON bundles in S3 and retrieve the relevant components at runtime.	Simple storage model, minimal database administration, content always source-of-truth JSON.	Slow lookup times, no selective reads, no query patterns, high latency risk, complex caching required, not suitable for real-time triage, rollback burden shifted into application logic.
Option 5: Single-table DynamoDB with Domain-mapped objects	Use a single DynamoDB table keyed by version and entity type; store each node as a domain object aligned to the triage engine architecture. Version isolation and rollback via pointer change; content immutable.	Proven low-latency, scalable, aligns with DDD and hexagonal engine design, clean rollback, simple access patterns, meets API stateless constraints, version isolation guaranteed, supports incremental evolution, lighter operational footprint.	Requires strict discipline in domain model alignment, schema evolution must be additive, less flexible for ad-hoc reporting (requires export pipeline), engineers must understand key design and access patterns.

Option	Description	Pros	Cons
3a. Multi-table per entity	Separate tables (questions, outcomes, etc.) with version in keys	Per-entity ownership; some reporting ease	Cross-table hops; more IAM/IaC; drift risk
3b. Table-per-version (selected)	**One table per release**; lifecycle = active → supported → archived; export to S3; rehydrate when needed	**Hard isolation**; clean rollback; **map API/Lambda to table**; future SNOMED changes isolated	**Higher ops overhead** (tables/alarms/IAM); export/import pipeline; table-count governance
3c. Hybrid (3A with strict version PK)	Entity tables with version in PK	Strong isolation; fewer items per table	Still multi-table complexity; cross-entity hops

12.3.2 3.2 Recommendation

Adopt **Option 3B: DynamoDB table-per-version** with strict lifecycle management (active → supported → deprecated → archived), and integrate with an authoritative **Version Registry** to route API and database lookups based on request headers.

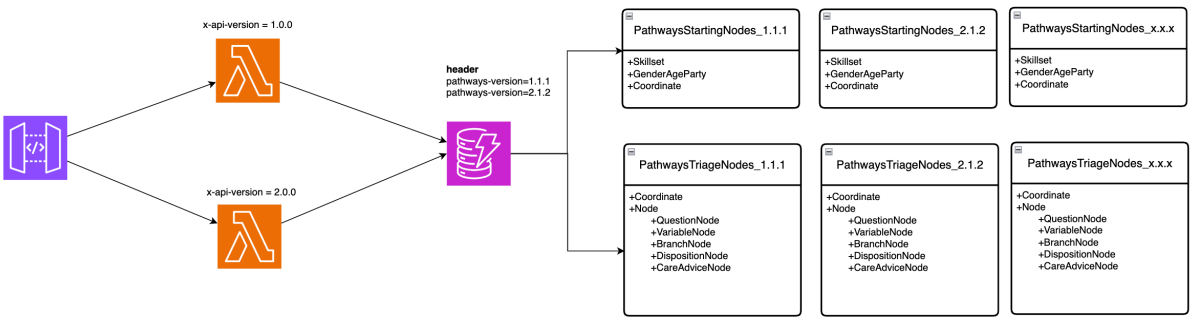
Within this model, the API will use **two explicit version headers**:

- `X-Pathways-Version` (selects the Pathways content version/table)
- `X-API-Version` (selects the API implementation version/Lambda alias)

Routing lambda validates both headers and consults the Version Registry to determine the correct table and Lambda alias.

No defaults are used in production. Both headers must be present on every request.

- **One DynamoDB table per Pathways content release** (e.g. P`athwaysStartingNodes-v1_1_1`)
- *On production, only a bounded set of tables are "live" at any time (e.g.* ***ACTIVE*** *+ limited* ***SUPPORTED*** *for supplier migration)*
- Future considerations will include
  - Older versions are **exported to S3** (checksum + manifest recorded) and **tables deleted** to control cost
  - If governance/audit requires, a historical version is **rehydrated** from S3 into a new table with the same version tag
  - For now, all lambdas and all pathway versions will be available for test in the pre-prod environment
- A **version registry** (Parameter Store or a small control table) defines:
  - **ACTIVE** table (default)
  - **Routing map:** x`-pathways-version` → **table name**
- API remains **stateless**; clients manage state. The API always reads from the **table selected by registry + header**.
- Each version table stores the same domain-aligned entities as before; strict immutability applies within a table.



12.3.2.1 3.2.1 Version Routing with Dual Headers

The API uses two mandatory request headers in production:

12.3.2.1.1 `X-Pathways-Version`

- Identifies the Pathways content version
- Maps directly to a specific DynamoDB table
- Must refer to a table in state ACTIVE or SUPPORTED

12.3.2.1.2 `X-API-Version`

- Identifies the API implementation version
- Maps to a specific Lambda alias
- Supports parallel API versioning, canary releases, supplier-specific compatibility

12.3.2.1.3 Routing workflow

1. APIM receives the request and validates both headers
2. Lambda queries Version Registry
3. Registry resolves:
   - `X-Pathways-Version` → Pathways table name
   - `X-API-Version` → Lambda alias
4. API request is executed via the resolved Lambda alias, which reads content from the resolved table

No cross-version reads are permitted.

12.3.2.2 Error Handling

- Missing header: **400 Bad Request**
- Unknown version: **404 Not Found**
- Deprecated version (not ACTIVE or SUPPORTED): **410 Gone**
- Incompatible pairing (if enforced): **409 Conflict**

12.3.2.3 3.2.2 Table Keys and item shapes

Because version is encoded in the **table name**, primary keys focus on entity identity and traversal:

Key	Purpose	Example
`PK`	Entity identity (coordinate or id)	`PA124.1500`
`SK`	Type-qualified identity for ordered reads where needed	`question#PA124.1500`

Recommended pattern (keeps domain clarity and predictable access):

- **Type in SK prefix** (e.g. `question#`, `outcome#`, `variable#`, `startnode#...`)
- **Coordinate/identifier** remains canonical for traversal

12.3.2.4 3.2.3 Typical entities

- `question#<coordinate>`
- `branch#<coordinate.variant>`
- `variable#<id>`
- `outcome#<code>`
- `startnode#<skillset>#<demographic>#<source>`
- `meta#version` (release metadata inside the table for completeness)

12.3.2.5 3.2.3 Lifecycle Management

Each table goes through a defined lifecycle:

- ACTIVE
- SUPPORTED
- DEPRECATED
- ARCHIVED

Deprecated versions are exported to S3 with manifest and checksum, then deleted.
Rehydration creates a new table from S3 export and restores alarm/IAM configuration.

A Version Registry records:

- Current ACTIVE version
- Supported versions
- Mapping of API versions to Lambda aliases
- Mapping of Pathways versions to table names

12.3.2.6 3.3 Rationale

- **Strict boundaries:** Physical isolation eliminates cross-version reads and simplifies governance conversations (“that outcome came from *that* table”).
- **Future SNOMED changes:** New schema or attributes can land in **new version tables** without impacting historical versions or their safety case.
- **Routing control:** The **version registry** maps `X-Triage-API-Version` and **Lambda aliases** to **specific tables**, making parallel-run and phased deprecation straightforward.
- **Rollback:** Change the registry pointer back to a prior table; no data mutation.
- **Audit:** Table name + request UUID provide an unambiguous audit trail.

With strict lifecycle, export-archive, and rehydration, plus version-aware routing that maps API/Lambda versions to content tables. This provides strong isolation, clean audit lineage, safe evolution for SNOMED changes, and controlled supplier coexistence.

12.4 4. Consequences

Positive

- Deterministic engine traversal using domain objects
- Efficient stateless execution path
- Strong version isolation and rollback capability
- Proven low latency under Alpha conditions
- Supports clinical audit trails via UUID correlation
- Hard isolation and clean audit lineage per version
- Simple rollback (registry pointer change)
- Safe evolution for SNOMED CT or TOM extensions

Negative

- More AWS resources to manage (tables, alarms, IAM, PITR)
- Requires **export-to-S3** and **import/rehydration** automation
- Observability must include **table name** in logs and traces
- Capacity and cost fragmented across tables

Out of Scope / Not Supported

- Free-form JSON storage
- In-place mutation of clinical content
- Arbitrary lookup queries not supported by engine domain model

12.5 5. Compliance

- Fully aligned with:
  - ADR-010 (API Architecture): stateless, deterministic traversal
  - ADR-011 (API Versioning): explicit routing to tables via registry
  - ADR-012 (Content Versioning): immutability per release
  - ADR-017 (Pathways Loading): create/load/validate/register/publish
  - ADR-018–020 (Observability/Alerts/Metrics): per-table metrics and routing logs
  - ADR-021 (Release): dual stream; registry updates are governed
  - ADR-022 (DR/BCP): PITR on ACTIVE/SUPPORTED; **rehydration drill** part of DR testing
- Complies with NHS DSPT and NCSC principles (data minimisation, encryption, audit).
- Supports clinical safety governance through deterministic, versioned outcomes.

12.6 6. Actions

- Implement DynamoDB table and index structure using IaC templates.
- Document schema and entity relationships in the API specification.
- Develop ingestion process to populate versioned entities (ADR-017).
- Validate latency and scaling under national-load performance tests.
- Prepare schema diagram for design documentation and Confluence.

13 ADR-API-013: Loading Pathways into the Triage Database

Date	01 Sept 2025
Status	IN PROGRESS
Deciders	Engineering, Architecture, Technical Review and Governance
Significant	
Owners	Tharack Ahmed
Decision	
Decision waiver	

13.1 Context

The **Triage API** requires a database to store and serve triage pathways. These pathways originate from the NHS Pathways dataset, which is updated periodically. We need a robust, scalable, and maintainable mechanism to **import triage pathway data into a database** to keep the API aligned with the latest clinical updates.

For the **Alpha phase**, the database chosen is **AWS DynamoDB**. This is an **assumption** for now; the database solution will be reviewed and validated during the **Beta phase**. The design therefore needs to be **database agnostic**, ensuring the data ingestion mechanism can adapt to different underlying database technologies if required.

Currently, NHS Pathways provides triage data through:

- CSV releases (primarily used by other suppliers)
- The **Pathways Authoring Tool** database
- The **Pathways Release Service**

Regardless of the source selected, we will need to **transform the data into a database-compatible schema** and **automate ingestion** in a way that minimises tight coupling to any single database technology.

The ingestion process must ensure that:

- The content is valid and complete.
- The release is linked to a unique version identifier (ADR-012).
- The data conforms to the data model defined in ADR-016.

- The operation is reproducible, auditable, and reversible.

The process must protect against partial loads, schema drift, and version contamination while providing a clear governance trail for clinical and technical assurance.

13.2 Business Drivers

1. Guarantee integrity, completeness, and traceability of every content load.
2. Provide automated, auditable ingestion supporting versioning and rollback.
3. Align content ingestion with governance, clinical safety, and technical release workflows.
4. Minimise downtime and ensure continuous API availability.
5. Enable validation and alerting on ingestion failures.

13.3 Decision

We need to choose a **primary data source** for importing triage pathways into a database. Once chosen, we will implement a **data ingestion pipeline** that automates ingestion and transformation, while remaining database agnostic.

13.3.1 Assumptions

- AWS DynamoDB is the **assumed database for Alpha**.
- The **Beta phase may select a different database technology** (e.g. relational or graph-based), depending on scale, query performance, and architectural needs.
- NHS will continue to release triage pathways in CSV and/or through the Release Service.
- API consumers will expect **consistent, standardised pathway data**, regardless of the underlying database technology.

13.3.2 Drivers

- Release Alignment – Staying synchronised with official NHS Pathways releases
- Reliability – Ensuring consistent and accurate triage data
- Automation – Minimising manual steps
- Scalability – Supporting larger datasets and future updates
- Database Independence – Avoiding design decisions that lock us into one technology
- Complexity – Reducing implementation and operational overhead
- Cost – Leveraging existing platform services where possible

13.3.3 Criteria

- Pathways are ingested into the database with no data loss or corruption

- The pipeline can be reconfigured to target a different database with minimal changes
- Ingestion process is automated, monitored, and requires minimal manual intervention
- System can handle increasing data volumes and release frequency

13.3.4 Options

13.3.4.1 Option 1 — Manual CSV Import (from NHS Pathways Releases)

Use the CSV files provided in periodic NHS Pathways releases, transform them into a generic structured format (e.g. JSON or SQL inserts), and bulk-load them into the database via the ingestion pipeline.

Pros:

- Simple and low initial setup effort
- Aligns with current NHS supplier processes
- Minimal dependency on external APIs

Cons:

- Manual intervention required for each release
- Delays possible when NHS releases updates
- Higher chance of human error
- Less suitable if update frequency increases

13.3.4.2 Option 2 — Direct Integration with Pathways Authoring Tool Database

Connect directly to the Pathways Authoring Tool database, extract the latest pathway data as CSV or JSON, transform it, and load it into the target database.

Pros:

- Immediate access to up-to-date data
- Eliminates dependency on CSV release schedules
- Greater control over extraction

Cons:

- Access may be unsupported or restricted
- Tight coupling to an internal system increases maintenance risk
- Security and access control considerations
- Higher operational complexity

13.3.4.3 Option 3 — Integration with the Pathways Release Service

Use the **Pathways Release Service** API to fetch the latest pathways programmatically and automatically load them into the target database using the ingestion pipeline.

Pros:

- Automates ingestion of new pathway versions
- Reduces manual intervention
- Likely to be the most future-proof if officially supported

Cons:

- Dependent on Release Service availability and SLA
- Requires engineering effort to integrate
- Potential limitations in API data formats or features
- May not be cost-effective to upgrade API to modern security and standards

13.3.4.4 Database-Agnostic Ingestion Pipeline (Required Implementation Layer)

This is **not a data source option** but a **necessary layer** to automate ingestion and transformation into the chosen database, whether AWS DynamoDB (Alpha), PostgreSQL, or another database in Beta.

Key principles:

- Ingestion pipeline outputs an intermediate data format (e.g. JSON or staging tables)
- Transformation layer should be loosely coupled with the database
- Database writes abstracted (e.g. through a data access layer)
- Supports switching target databases without re-engineering the source integration

Example implementation for Alpha:

- Orchestration: AWS Lambda
- Storage: Amazon S3 for raw CSVs (or equivalent)
- Transformation: AWS Glue or custom scripts
- Database writer: Adapter for DynamoDB (Alpha), extendable to other DBs
- Monitoring: CloudWatch or equivalent

Option	Description	Pros (For)	Cons (Against)	Recommendation / Rationale
Option 1 – Manual content loading (scripts, database inserts)	Human-triggered ingestion of Pathways data into the database.	Simple initially; no additional tooling required.	High human error risk; inconsistent loading times; poor auditability; not scalable.	Chosen tactical option
Option 2 – Scheduled batch ingestion	Automated periodic jobs (e.g. nightly) to load Pathways updates.	Predictable timing; low operational complexity.	Delays between release and availability; harder rollback; difficult to coordinate with release pipeline.	Rejected – insufficient responsiveness and governance alignment.
Option 3 – Controlled, version-aware ingestion pipeline	API- or event-triggered ingestion pipeline that validates, versions, and loads Pathways content automatically.	Automated, auditable, repeatable; ensures consistency with versioning (ADR-012); integrates with release process (ADR-021).	Higher engineering complexity; requires ongoing pipeline maintenance.	Selected – best supports reproducibility, governance, and audit requirements while reducing operational risk. Chosen Strategic solution

Criteria	Option 1 CSV Import	Option 2 Authoring Tool DB	Option 3 Release Service
Release Alignment	High – updates depend on CSV release schedules	High – direct from authoring source	High – aligned with official releases
Automation Potential	Low – manual process required	Medium – possible but complex	High – built-in API support
Implementation Effort	Low	Medium to High	Medium
Operational Overhead	Medium – manual CSV handling required	High – maintenance-heavy	Low – automated ingestion
Scalability	Limited if update frequency increases	Moderate, depends on DB integration	High – designed for large updates

Criteria	Option 1 CSV Import	Option 2 Authoring Tool DB	Option 3 Release Service
Security & Access	Secure – NHS provides packaged CSVs	Risky – requires direct DB access	Controlled via supported API
Future-Proofing	Limited – CSV process may change	Low – dependent on internal system stability	High – preferred long-term solution
Dependency Risk	Low – NHS CSV releases stable	High – tied to authoring DB availability	Medium – depends on Release Service SLA
Best Fit For	Short-term bootstrapping	Experimental or internal tooling	Strategic, automated ingestion

13.3.5 Recommendation (Tactical and Strategic)

Given that the **Pathways Release Service (PRS)** is currently at risk of being retired and does not yet provide a stable or guaranteed interface for long-term automation, the ingestion approach is updated as follows:

13.3.5.1 Tactical (Immediate) Approach

- **Use Manual CSV Import (Option 1) as the primary ingestion mechanism** for the immediate term.
- CSV files will be copied manually from the NHS Pathways distribution into the ingestion pipeline's S3 landing area.
- The ingestion pipeline will still run in **automated mode** after the CSVs are deposited (schema validation, transformation, load, and verification).
- Manual copying is treated as a **controlled, auditable operational step**, not a fully manual ingestion.

13.3.5.2 Strategic (Future) Approach

- **Integration with the Pathways Release Service (Option 3) or a version of automated release** remains the intended *long-term* solution.
  - When PRS is confirmed to be production-ready and not at risk of deprecation, the ingestion pipeline will migrate to fully automated ingestion via PRS.
  - Or Authoring Tools workstream finds an alternative release process
- CSV import will remain as a fallback mechanism for resilience and contingency.

This ensures service continuity, clinical safety, and predictable ingestion despite uncertainty around PRS availability.

13.3.6 Rationale

The tactical choice of using manual CSV import is driven by:

1. **PRS instability risk**
   PRS is at risk of being retired or re-platformed; relying on it for an Alpha/Beta ingestion process creates delivery and operational risk.
2. **Continuity of supplier process**
   CSV releases are currently the primary mechanism used by other NHS Pathways suppliers and have a **proven, stable operational footprint**.
3. **Low-risk fallback**
   CSVs require human placement but maintain the **lowest dependency risk** while still supporting automation downstream.
4. **Clinical safety**
   Manual placement of CSVs allows explicit checks before ingestion, ensuring the content aligns with governance and clinical safety requirements.
5. **Strategic alignment for later phases**
   When PRS stabilises or re-emerges in a supported form, the ingestion pipeline—being database-agnostic and source-agnostic—can switch to API-based ingestion with minimal change.

This approach ensures Alpha and Beta delivery are not blocked while preserving a high-automation future path.

13.3.7 Tactical Mitigation: Manual CSV Placement Process

Due to PRS uncertainty, a **repeatable operational runbook** will be introduced:

1. **NHS Pathways CSV release is published**
2. **Operational engineer copies CSV bundle into S3 landing bucket**
   - This step is logged and timestamped
   - Version ID is manually validated against governance records
3. **Ingestion pipeline triggers automatically**
   - Extraction
   - Structural validation
   - Transformation to domain model format
   - Load into version-specific DynamoDB table
4. **Ingestion verification**
   - Item count checks
   - Checksum comparison
   - Schema compliance
5. **Version registry update**
   - Promotion to ACTIVE or SUPPORTED states (with governance approval)

13.3.7.1 Benefits

- No dependence on PRS availability
- Fully automated after initial manual step
- Maintains clean versioning (see ADR-012) and version table isolation (ADR-016)
- Minimises risk of data corruption or misaligned releases

13.4 Consequences

Positive:

- Clear separation between data source selection and data ingestion pipeline
- Database-agnostic design supports flexibility in Beta
- Scalable, automated ingestion with monitoring
- Reduced risk of rework if database choice changes

Negative:

- Requires discipline in designing the pipeline to remain database-agnostic
- Upfront development effort required
- Ongoing monitoring and operational support needed
- Reliance on manual CSV placement introduces an operational step that must be governed carefully to avoid delays or human error, though this risk is mitigated by automation of all subsequent steps.

13.5 Compliance

- **Aligned with:**
  - ADR-012 (Pathways Content Versioning)
  - ADR-015 (Database Choice)
  - ADR-016 (Data Model)
  - ADR-018 (Monitoring and Observability)
  - ADR-021 (Release Process)
- **Compliant with:** NHS DSPT, DCB0129/0160 clinical safety guidance, and NHSE governance standards.

13.6 Notes

- Direct integration with the Authoring Tool (Option B) is considered high risk and unlikely to be supported long-term.
- CSV ingestion (Option A) is the safest short-term fallback but not scalable.

- The Release Service (Option C) appears to be the most strategic choice if it is production-ready.
- The ingestion pipeline should include version control for pathways, so previous releases can be replayed if required.

13.7 Actions

1. Confirm availability and maturity of the **Pathways Release Service**.
2. Build a **prototype ingestion pipeline** that supports both CSV and Release Service inputs.
3. Implement **database-agnostic adapters** starting with AWS DynamoDB for Alpha.
4. Document assumptions and revisit database choice for **Beta phase**.
5. Ensure monitoring, logging, and audit trail are in place from the outset.

14 ADR-API-014: Monitoring and Observability Tooling

Date	24 Sept 2025
Status	Draft
Deciders	NHS England, DUEC SMT, Architecture & Engineering
Significant Owners	SaET Team
Decision	Adopt a hybrid monitoring and observability approach using AWS-native services for engineering needs, integrated with Splunk/Odin for governance and security oversight

14.1 1. Context

The triage API underpins critical national services (NHS 111 and 999), where service availability, data integrity, and performance directly affect patient safety and public trust. Comprehensive observability is required not only for the API itself but also for supporting components such as data ingestion and version loading.

This ADR builds upon and must be read alongside:

- **ADR-API-010:** Defines the underlying AWS infrastructure and security model that observability will monitor.
- **ADR-API-012:** Defines the data model and key entities that must be visible in metrics and logs.
- **ADR-API-013:** Defines the ingestion process for loading Pathways content, which must be closely monitored to ensure completeness, timeliness, and correctness.

Observability must address three main domains:

- **Operational monitoring** for engineering teams.
- **Governance observability** for NHS boards and programme oversight.
- **Security and audit visibility** for compliance and investigations.

14.2 2. Business Drivers

- Ensure operational transparency and clinical safety through complete traceability.
- Enable proactive detection and remediation of incidents before user impact.
- Support post-incident analysis and governance reporting.
- Provide a single source of truth for performance and reliability data.
- Integrate with NHS corporate security, SIEM, and compliance systems.

14.3 3. Decision

14.3.1 3.1 Options

Option	Description	Advantages	Disadvantages
Option 1: AWS-only observability	Use CloudWatch, X-Ray, and CloudTrail exclusively for metrics, traces, and logs.	• Simplest integration. • No data egress. • Full AWS native automation.	• Limited cross-system correlation. • Retention limits for long-term audit. • No direct link to NHS SIEM for governance view.
Option 2: Enterprise SIEM only (Splunk/ Odin)	Forward all telemetry to NHS SIEM platform; minimal AWS native use.	• Centralised NHS operations visibility. • Long-term retention and SOC integration.	• Higher cost and latency. • Complex to integrate directly with AWS services. • Limited real-time insight for engineering teams.
Option 3: Hybrid observability (selected)	Use AWS native for real-time metrics/tracing; forward logs and key events to Splunk/Odin for enterprise correlation and audit.	• Best of both models. • Real-time visibility for developers. • Centralised governance, compliance, and audit view. • Scalable and cost-balanced.	• Integration complexity. • Requires disciplined data filtering and retention strategy.

14.3.2 3.2 Recommendation

Adopt **Option 3 – Hybrid observability** has been selected.
It balances engineering needs (low-latency metrics and traces) with governance needs (cross-system audit and compliance).
AWS CloudWatch and X-Ray provide near-real-time telemetry; Splunk or Odin provides long-term, secure visibility for operational governance and incident analysis.

Layer	Technology	Purpose
Metrics and Alarms	AWS CloudWatch	Application, API Gateway, and DynamoDB metrics; alarms and SLAs.

Layer	Technology	Purpose
Distributed Tracing	AWS X-Ray	End-to-end transaction traces; latency and dependency visualisation.
Logging (runtime and audit)	CloudWatch Logs → Kinesis → Splunk/Odin	Centralised logs with UUID correlation and retention per governance policy.
Compliance / Security events	AWS CloudTrail + Security Hub	API activity tracking, configuration changes, security posture.
Dashboards and Reporting	Grafana / Splunk dashboards	Unified view of operational and clinical-safety metrics.

14.4 4. Consequences

- Two-tier monitoring: AWS for live engineering dashboards; Splunk/Odin for strategic reporting.
- Data forwarding adds cost and complexity but ensures compliance with NHS standards.
- Requires UUID propagation across API, APIM, and downstream logs for correlation.
- Enables 24/7 alerting and SIEM-driven incident response.

14.5 5. Compliance

- **Aligned with:**
  - ADR-API-006(API Architecture)
  - ADR-API-010(AWS Infrastructure and Security)
  - ADR-API-011 (Database Choice)
  - ADR-API-015 (Metrics and Alerts)
  - ADR-API-018 (DR & Business Continuity)
- **Compliant with:** NHS DSPT, NCSC cloud logging principles, and NHS security-operations tooling (Splunk/Odin).

14.6 6. Actions

- Configure CloudWatch and X-Ray instrumentation in all Lambda/ECS components.
- Set up Kinesis Firehose to forward structured logs to Splunk/Odin.
- Implement UUID correlation (`X-Triage-Request-ID`) in all logs.
- Develop governance dashboards showing availability, latency, error rate, and ingestion health.
- Validate observability through resilience and DR tests (ADR-API-018).

15 ADR-API-015: Alerts

Date	24 Sept 2025
Status	Draft
Deciders	NHS England, DUEC SMT, Architecture & Engineering
Significant Owners	SaET Team
Decision	Implement a tiered alerting strategy distinguishing between operational issues, governance incidents, and clinical safety risks

15.1 1. Context

The Triage API is a clinically critical platform supporting NHS 111 and 999. Operational failure could directly affect patient safety and service delivery.

To ensure timely identification and resolution of issues, automated alerting is required for operational, performance, and security events. Alerts must be structured, auditable, severity-driven, and routed to NHS operational support functions.

Alerting must integrate with cloud monitoring, enterprise logging, and NHS service management processes, ensuring the system reacts appropriately to degradation and incidents without generating unnecessary noise.

This ADR defines how alerts are generated, classified, routed, escalated, and reviewed.
It does not define metrics or thresholds — those are specified in **ADR-API-016: Metrics**.

15.2 2. Business Drivers

- **Clinical safety:** Ensure critical incidents are detected and escalated quickly.
- **Governance:** Provide oversight of significant service-affecting events.
- **Operational effectiveness:** Prevent alert fatigue by filtering noise.
- **Compliance:** Meet NHS Cyber and DSPT monitoring requirements.

15.3 3. Decision

15.3.1 3.1 Options

Option	Description	Pros (For)	Cons (Against)	Recommendation / Rationale
Option 1 – Flat alerting model	All alerts treated equally without severity levels.	Quick to implement; simple routing.	Alert fatigue; poor prioritisation; slow response to critical incidents.	Rejected – unsafe and unmanageable at scale.
Option 2 – Tiered severity-based alerting	Multi-level severity (e.g., P1–P4) with distinct escalation paths and SLAs.	Prioritised response; reduces noise; aligns with clinical and operational needs.	Requires tuning and governance agreement.	Selected – ensures critical incidents receive rapid attention while minimising noise.
Option 3 – Governance -only alerts	Alerts triggered only for major governance events.	Low noise; easy oversight.	Poor operational visibility; delayed response; unsafe for clinical context.	Rejected – operationally and clinically inadequate.

15.3.2 3.2 Recommendation

Adopt Option 2 – Tiered alerting model.

Alerts will be generated automatically from agreed thresholds, routed through AWS native tooling and onward to NHS incident management systems.
A severity-based model will govern routing and response, aligned with NHS operational policies.

Alerts must include the correlation UUID to support audit and investigation.

Key decisions:

- Alerts generated based on metrics defined in ADR-API-016
- AWS CloudWatch → SNS used for primary alerting
- Alerts forwarded to Microsoft Teams and ServiceNow
- Security and critical alerts forwarded to Splunk/Odin for SOC visibility
- Structured severity classification (P1–P4) and can be found here: API Service Management[39]
- Automatic incident creation in ServiceNow for P1 and P2 events
- Alerts reviewed regularly to manage noise and maintain effectiveness - Triage API alert logs[40]

This approach provides a reliable, compliant, and real-time alerting framework aligned with NHS governance, cyber security, and operational practices.
It ensures fast response while maintaining audit trail and CSOC integration.

39. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1237096858/API+Service+Management#APIServiceManagement-SeverityLevels
40. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1237098806/Triage+API+alert+logs

15.4 4. Consequences

- **Positive**
  - Rapid detection and escalation of incidents
  - Aligns with NHS operational processes and SOC requirements
  - Traceability via UUID across logs and alerts
  - Supports proactive service management and clinical safety monitoring

  Negative
  - Requires maintenance of alerting configurations and runbooks
  - Needs periodic tuning to prevent alert fatigue
  - Dependency on AWS and enterprise SIEM integration

15.5 5. Compliance

- Supports NHS DSPT and NHS Cyber Security monitoring standards.
- Aligns with NHS principle of transparency in incident reporting.

15.6 6. Actions

- Configure CloudWatch → SNS → Teams and ServiceNow
- Implement Splunk/Odin forwarding for audit and security alerts
- Define and publish alerting playbooks

16 ADR-API-016: Metrics

Date	24 Sept 2025
Status	Draft
Deciders	NHS England, DUEC SMT, Architecture & Engineering
Significant Owners	SaET Team
Decision	Define and implement a metrics catalogue covering operational, governance, and clinical safety dimensions

16.1 1. Context

The triage API is a safety-critical service underpinning 111 and 999 operations. Metrics are essential for demonstrating system performance, supporting clinical safety cases, enabling governance reporting, and validating supplier trust.

Metrics must provide visibility not only into the **API runtime** but also into the **data model (ADR-API-012)**, **Pathways ingestion process (ADR-API-013)**, **observability platform (ADR-API-014)**, and **alerting strategy (ADR-API-015)**. This ensures that the full lifecycle — from content ingestion through triage decision generation — is measurable and auditable.

This ADR builds upon:

- **ADR-API-010:** Infrastructure and security model — metrics must measure system-level health.
- **ADR-API-012:** Data model — metrics must capture data completeness and integrity.
- **ADR-API-013:** Ingestion — ingestion performance, success, and latency metrics are critical.
- **ADR-API-014:** Monitoring — defines how metrics will be surfaced and visualised.
- **ADR-API-015:** Alerts — metrics must underpin severity definitions and thresholds.

16.2 2. Business Drivers

- **Clinical safety:** Evidence that the API consistently produces safe, reproducible outcomes.
- **Resilience:** Demonstrate system stability and response times for 111/999 services.
- **Governance:** Provide transparency to NHS England boards and compliance auditors.
- **Supplier trust:** Demonstrate reliability of the national triage API.

16.3 3. Decision

16.3.1 3.1 Options

Option	Description	Pros (For)	Cons (Against)	Recommendation / Rationale
Option 1 – Operational metrics only	Track latency, uptime, error rate, etc.	Simple, low overhead; quick to deploy.	Limited governance and clinical insight; ingestion and data quality not tracked.	Rejected – insufficient for NHS governance and safety cases.
Option 2 – Comprehensive metrics catalogue	Covers operational, ingestion, data integrity, governance, and clinical safety metrics.	Holistic visibility; enables proactive improvement; informs alerts and audits.	Requires more design and ongoing governance oversight.	Selected – essential for clinical safety, governance compliance, and supplier trust.
Option 3 – Governance metrics only	Track version adoption, release frequency, and SLA compliance.	Useful for board-level reporting.	Misses operational issues; no clinical safety metrics.	Rejected – incomplete picture of system performance and safety.

16.3.2 3.2 Recommendation

Adopt Option 2 – Comprehensive metrics catalogue.

- Define metrics collaboratively with engineering, governance, and clinical safety leads.
- Ensure ingestion metrics (ADR-API-013) and data model metrics (ADR-API-012) are captured and visualised.
- Integrate metrics into dashboards defined in ADR-API-014.
- Use metrics to inform alert thresholds and severity triggers defined in ADR-API-015.

Metrics and KPIs to be measured can be found here: Triage API Metrics and KPIs[41]

16.4 4. Consequences

- Positive: Holistic visibility, alignment across stakeholders, builds trust.
- Negative: Requires governance overhead and ongoing maintenance.

41. https://nhsd-confluence.digital.nhs.uk/spaces/NPR/pages/1196017184/Triage+API+Metrics+and+KPIs

16.5 5. Compliance

- Supports DSPT, NHS Cyber Security, and ISO 27001.
- Aligns with NHS clinical safety case requirements for reproducibility and audit.

16.6 6. Actions

- Define a metrics catalogue including ownership, thresholds, and review frequency.
- Implement metric collection across application, ingestion, and infrastructure layers.
- Integrate metrics into dashboards and governance reports (ADR-018).
- Review metrics quarterly and update based on usage patterns, incidents, and safety findings.

17 ADR-API-017: Triage API - Release Process

Date	24 Sept 2025
Status	**WIP**
Deciders	Architecture & Engineering
Significant Owners	SaET Team
Decision	Implement a gated release process with automated pipelines, governance approvals, and supplier communication

17.1 1. Context

The Triage API underpins clinical triage outcomes for 111 and 999 services.
Any change to the API codebase, underlying infrastructure, or clinical content (Pathways data) must follow a **controlled, auditable, and reversible release process**.

Releases must:

- Comply with NHS England technical and clinical governance frameworks.
- Support zero-downtime deployment.
- Maintain backward compatibility across versions (ADR-API-007).
- Provide rollback capability and clear communication to downstream suppliers.

Both **API software** and **Pathways content releases** are in scope, though they are managed through separate but coordinated pipelines

17.2 2. Business Drivers

- Ensure safe, auditable, and consistent deployment of new versions.
- Ensure operational continuity and reduce deployment risk.
- Enable predictable release scheduling and governance sign-off.
- Provide transparent version traceability for clinical and technical auditing.
- Support modern CI/CD practices within NHS assurance constraints.
- Maintain operational stability and minimise downtime.
- Support rapid iteration within controlled governance boundaries.
- Comply with NHS change management and clinical safety processes.
- Provide traceability from release artefact to deployed version.

17.3 3. Decision

17.3.1 3.1 Options

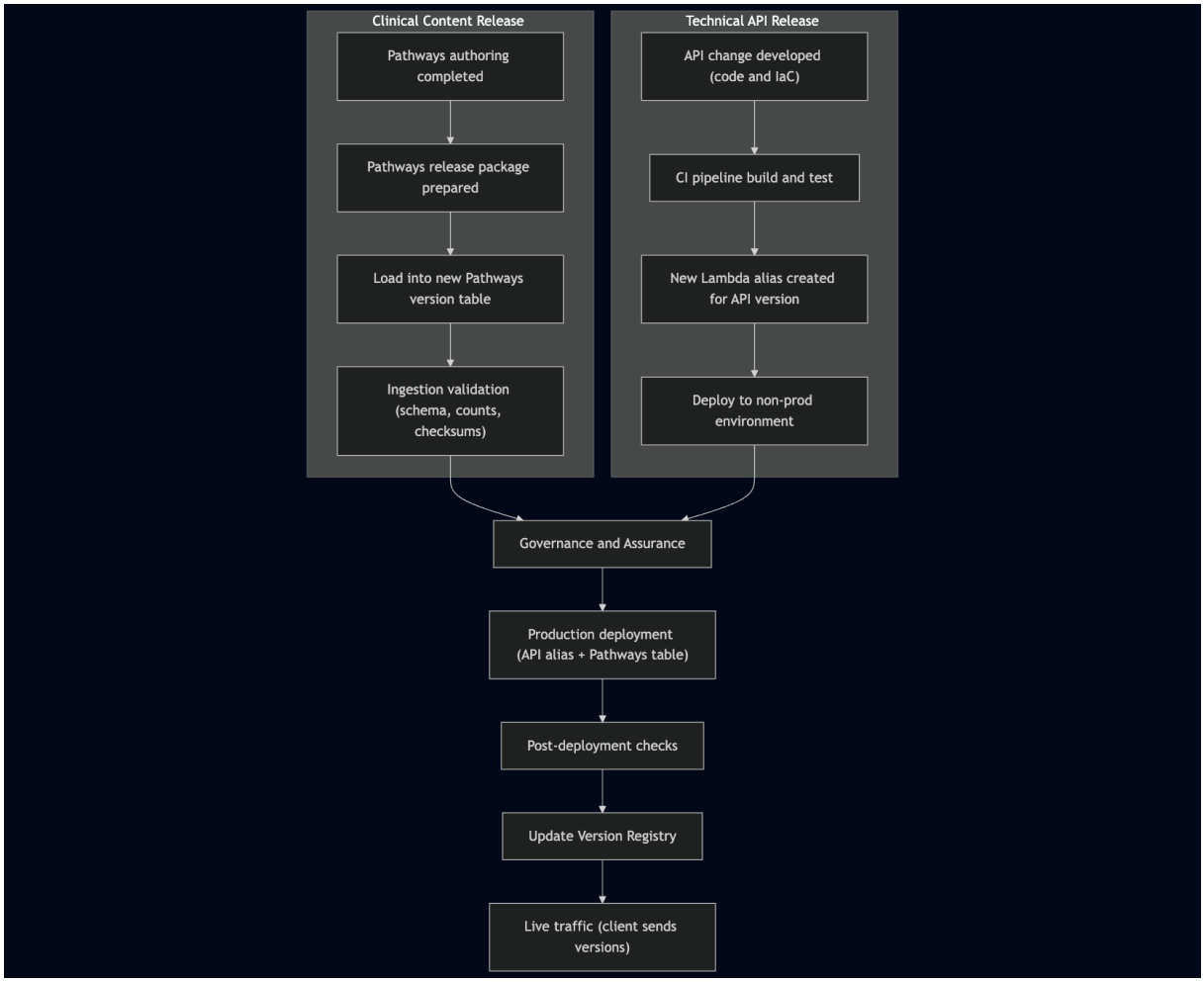
Option	Description	Advantages	Disadvantages
Option 1: Manual release with CAB approval only	Releases performed manually via scripts or console, with CAB approval as sole control.	• Simple to initiate. • No major tooling dependencies. • Familiar to existing NHS ops teams.	• Error-prone and slow. • Poor reproducibility and rollback. • High risk of configuration drift. • No automated validation or test gating.
Option 2: Single unified CI/CD pipeline (code + content)	One pipeline manages both technical and clinical content releases.	• Centralised view of all changes. • Simplified automation surface.	• Governance and clinical sign-offs cannot easily be decoupled. • Clinical content cadence differs from technical release cadence. • Increased regression risk if content changes accompany code deploys.
Option 3: Two-stream CI/CD with governance checkpoints (selected)	Separate automated pipelines for technical and content releases, converging at governance approval and production deployment.	• Aligns with NHS governance structure. • Enables independent validation and scheduling. • Supports automated rollback per stream. • Integrates cleanly with ADR-012 and ADR-017. • Reduces clinical risk while maintaining agility.	• More initial setup complexity. • Requires strong release coordination. • Additional governance documentation overhead.
Option 4: Fully automated continuous deployment (no CAB)	Continuous deployment directly from main branch once tests pass.	• Maximum agility. • Minimal manual effort.	• Non-compliant with NHS governance and clinical safety sign-off requirements. • Unacceptable risk for patient-facing system.

17.3.2 3.2 Recommendation

The Triage API release process will follow a **two-stream governance-controlled CI/CD model**:

1. **Technical release pipeline** – for code, infrastructure, and configuration changes.
2. **Clinical content release pipeline** – for new or updated Pathways data.

Both streams will use version-controlled Infrastructure-as-Code (IaC) and content repositories, with automated testing, validation, and deployment under the oversight of NHS governance boards. It provides automation and repeatability while respecting the clinical safety and technical assurance controls required by NHS England.



17.3.3 3.3 Technical Release Pipeline

Stage	Description	Key Tools / Controls
Code Commit	Developers commit code changes to main branch.	GitHub / GitLab; protected branches.
Build & Test	Automated unit, integration, and performance tests.	AWS CodeBuild / Jenkins; artefact versioning.
Security & Compliance	Code scanning and dependency checks.	SonarQube, Trivy, OWASP dependency scan.
Pre-Production Deploy	Deploy to staging; smoke tests and user acceptance.	AWS CodePipeline / Terraform IaC.
Governance Sign-off	Architecture and clinical-safety boards approve release.	Governance workflow / JIRA / ODP linkage.
Production Deployment	Automated deployment via blue/green or canary pattern.	AWS CodeDeploy / Lambda alias versioning.
Post-Release Validation	Confirm version alignment and operational stability.	Monitoring dashboards (ADR-018).

17.3.4 3.4 Clinical Content Release Pipeline

Stage	Description	Key Tools / Controls
Content Approval	Pathways content approved by authoring governance board.	Authoring system / Governance database.
Version Registration	New version ID allocated and validated (ADR-012).	??
Ingestion Trigger	Event-driven load into DynamoDB (ADR-017).	AWS Step Functions / Lambda.
Validation & Checksums	Schema validation, completeness checks.	??

Stage	Description	Key Tools / Controls
Activation	Version marked active; older version deprecated.	API control flag in DynamoDB metadata.
Post-Release Review	Metrics and audit verification post-deployment.	Splunk/Odin dashboards.

17.3.5 3.5 Release Governance and Assurance

- Release Governance and Assurance can be found here: Triage API - Release Governance (see page 99)

17.4 4. Consequences

- Strong audit trail but longer lead time due to governance sign-off.
- Safe rollback and accountability across technical + clinical domains.
- Predictable cadence and controlled risk profile.
- Greater coordination required between dev, ops, and governance teams.

17.5 5. Compliance

- **Aligned with:** ADR-API-007 (Versioning), ADR-API-008 (Content Versioning), ADR-API-010 (Infrastructure & Security), ADR-API-013 (Pathways Loading), ADR-API-014(Monitoring & Observability), ADR-API-018(DR & Business Continuity).
- **Compliant with:** NHS DSPT, DCB0129/0160, and NHSE Change Management Standards.

17.6 6. Actions

- Implement CI/CD pipelines for code and content using IaC.
- Define governance approval workflow in JIRA integrated with CAB.
- Automate release validation and post-release checks.
- Maintain release dashboard showing version lineage and approval status.
- Conduct periodic rollback drills and governance audits.

17.7 7. References

- Service Management: LSDT-760 - NHS Pathways Triage API[42]

42. https://nhsd-confluence.digital.nhs.uk/spaces/SDT/pages/1226692490/LSDT-760+-

17.8 Triage API - Release Governance

17.8.1 Release Governance

- All releases (technical or clinical) require **Change Advisory Board (CAB)** approval.
- Clinical content releases must include a clinical-safety sign-off.
- Release artefacts (ADRs, ODPs, test evidence) are stored in version control.
- Rollback procedures:
  - Technical rollback = re-deploy previous artefact.
  - Clinical rollback = switch `pathway_version_id` to previous active version.
- Each production deployment generates a release record in the governance register (linked to JIRA and ADR references).

17.8.2 Break Glass Process

- [ ] To be confirmed

+NHS+Pathways+Triage+API

18 ADR-API-018: Disaster Recovery and Business Continuity

Date	24 Sept 2025
Status	Draft
Deciders	Architecture & Engineering
Significant Owners	SaET Team
Decision	mplement a multi-AZ disaster recovery strategy with defined RTO/RPO, tested playbooks, and governance oversight

18.1 1. Context

The Triage API is a mission-critical service supporting NHS 111 and 999 triage decisions.
Its failure or prolonged outage could have **direct patient-safety and operational impacts** across the urgent and emergency care ecosystem.

The system must therefore meet NHS England's resilience requirements through a **comprehensive disaster-recovery (DR) and business-continuity (BC) strategy**.
This strategy must ensure availability, protect data integrity, and allow rapid restoration following incidents, while maintaining auditability and compliance with governance standards.

Key continuity challenges include:

- Loss of availability of AWS infrastructure components.
- Data corruption or unauthorised modification of Pathways content.
- Deployment or configuration failures rendering API unavailable.
- Upstream dependencies (e.g., APIM or authoring pipeline) becoming unavailable.

18.2 2. Business Drivers

- Maintain service availability to protect patient safety.
- Meet NHS-mandated recovery objectives for critical systems.
- Ensure data integrity and continuity of versioned Pathways content.
- Provide operational resilience and clear recovery procedures.
- Comply with DSPT, ISO 27001, and NHSE Resilience Assurance requirements.

18.3 3. Decision

18.3.1 3.1 Disaster Recovery Strategy Overview

Layer	Approach	RTO / RPO Target	Mechanism
Infrastructure	Active-active multi-AZ deployment.	RTO < 15 minRPO = 0	AWS Lambda / ECS auto-scaling across ≥2 AZs.
Database (DynamoDB)	Native multi-AZ replication with PITR (Point-in-Time Recovery).	RTO < 15 minRPO ≤ 5 min	Continuous replication, automated backups.
API Layer	Stateless; redeployable via IaC templates.	RTO < 30 min	Blue/green deployment rollback; IaC re-provision.
Content Ingestion Pipeline	Replayable event-driven process.	RTO < 60 min	Step Functions and SQS event replay.
Monitoring & Logging	Centralised, cross-region log storage.	N/A (continuous)	CloudWatch Logs → Splunk/ Odin.
Governance & Release Artefacts	Version-controlled, backed-up repositories.	RTO < 60 min	Git-based backups; mirrored repos.

18.3.2 3.2 Options

Option	Description	Advantages	Disadvantages
Option 1: Backup-only recovery	Daily backups of DynamoDB and configuration; manual restore in case of failure.	• Low cost. • Simple to maintain.	• High RTO (>24 hrs). • Significant clinical risk. • Non-compliant with NHS critical-service standards.
Option 2: Active-passive standby (multi-region)	Full secondary environment in different region, activated on failover.	• High resilience. • Geographic redundancy. • Minimal downtime during region-wide failure.	• High cost for standby. • Complex replication and failover management.

Option	Description	Advantages	Disadvantages
Option 3: Active-active multi-AZ (selected)	API and database deployed across multiple Availability Zones within same region, supported by automated IaC redeployment.	• Automatic fault tolerance. • Low RTO/RPO. • Fully automated recovery. • Aligned with AWS Well-Architected and NHS resilience standards.	• Does not protect against total region outage. • Requires careful testing and health-check automation.

18.3.3 3.3 Recommendation

Option 3 – Active-active multi-AZ with IaC-driven recovery has been selected.

The Triage API will adopt a **multi-AZ, resilient architecture** (see ADR-014) combined with a **tiered disaster-recovery strategy** encompassing data, application, and operational layers.
The approach will provide **automated recovery, regular testing, and documented procedures** for business continuity.

It provides the best balance between cost, recovery time, and operational simplicity while meeting NHS continuity requirements.

All components (API, data, and pipeline) are stateless or recoverable, allowing **automated redeployment in under 30 minutes** and near-instant data restoration through DynamoDB's multi-AZ replication and PITR.

18.3.4 3.4 Business-Continuity Procedures

- **Incident detection:** Triggered via monitoring alerts (ADR-018/019).
- **Failover process:** Health checks automatically route traffic to healthy AZ; manual escalation if regional issue.
- **Data integrity validation:** Post-recovery checksum verification of Pathways data.
- **Rollback:** Redeploy previous artefact or revert active `pathway_version_id`.
- **Communication:** Immediate notification to NHSE Operations and supplier stakeholders.
- **BCP documentation:** Maintained in the operational runbook, updated quarterly.

4. Consequences

- Additional operational and testing overhead to maintain DR readiness.
- Slightly increased cost due to multi-AZ and cross-region logging.
- Significantly reduced risk of patient-impacting downtime.

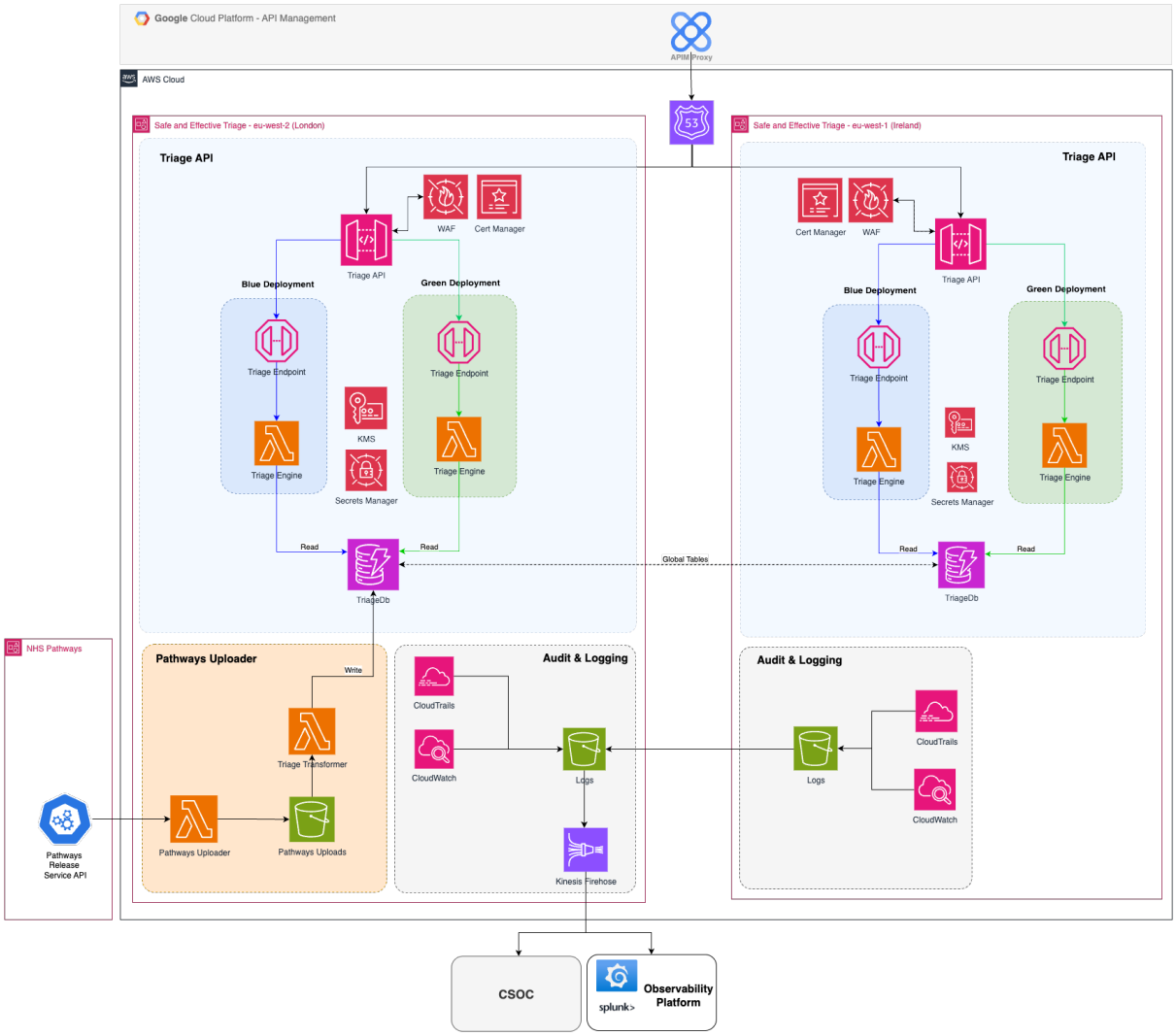
- Enables measurable compliance with NHS resilience requirements.

18.4 5. Compliance

- **Aligned with:**
  - ADR-API-006 (API Architecture)
  - ADR-API-010(AWS Infrastructure & Security)
  - ADR-API-014(Monitoring & Observability)
  - ADR-API-015(Metrics & Alerts)
  - ADR-API-017(Release Process)
- **Compliant with:** NHS DSPT, NCSC Cloud Resilience Principles, ISO 27001, and NHSE Business Continuity Standards.

18.5 6. Actions

- Implement IaC templates to automate DR redeployment across AZs.
- Configure DynamoDB PITR and daily cross-account backups.
- Schedule biannual DR and continuity simulation tests.
- Maintain operational BCP and DR runbooks in version control.
- Integrate DR test results with governance reviews and CAB records.


Google Cloud Platform - API Management
APIM Proxy
AWS Cloud
Safe and Effective Triage - eu-west-2 (London)
Safe and Effective Triage - eu-west-1 (Ireland)
Triage API
WAF
Cert Manager
Triage API
Blue Deployment
Green Deployment
Triage Endpoint
Triage Engine
KMS
Secrets Manager
Read
TriageDb
Global Tables
NHS Pathways
Pathways Release Service API
Pathways Uploader
Write
Triage Transformer
Pathways Uploads
Audit & Logging
CloudTrails
CloudWatch
Logs
Kinesis Firehose
CSOC
splunk>
Observability Platform

19 ADR-API-019: Infrastructure and role permissions

Date	25 Feb 2025
Status	`Proposed`
Deciders	`Architecture and Engineering`
Significant	
Owners	SaET Team
Decision	
Decision waiver	

19.1 Context

The Triage API and related components require a consistent, auditable approach to infrastructure and access management across environments. Current access patterns and environment separation need to be formalized to reduce risk, improve traceability, and support scalable delivery.

19.2 Drivers

- Least privilege and reduced exposure to unnecessary services or resources.
- Immutable, automated pipelines to ensure traceability and reduce human error.
- Role-based access with environment scoping to simplify access management.
- Centralized access management via infrastructure as code and SSO.
- Break-glass support for exceptional operational needs.
- Clear separation of environments and accounts for operational safety.

19.3 Decision

We will centralize access management in Terraform and AWS IAM Identity Centre, define permission sets per role and environment, and enforce least privilege across all accounts. We will use the following account structure for the Patient Triage API: DEV, TEST, PROD and MGMT Elevated access will be temporary and audited via the organizational TEAM process.

19.3.1 Decision details

19.3.1.1 Principles

- Least privilege access.
- Immutable pipelines via CI/CD.
- Role-based and environment-scoped permissions.
- Centralized access management via Terraform and SSO.
- Break-glass support via TEAM.

19.3.1.2 Architecture and tooling

- Terraform-based access control in the `aws-prod-sso-configs` repository.
- Permission sets aligned to role and environment.
- Azure AD groups linked to AWS IAM Identity Centre.

19.3.1.3 AWS accounts

Name	Purpose	Details
DEV - Developer	Development	Provisioned
TEST - Test	Testing new features; Integration testing with external systems; Testing new releases	Provisioned
MGMT - Management	Manage IAM policies attached to permission sets	Provisioned
PROD - Production	Live service environment	Provisioned
DR - Disaster Recovery	Immutable backups	To request

19.3.1.4 External environments

Name	Purpose	Hosting account
Sandbox	Open access, example API with canned responses.	TEST

Name	Purpose	Hosting account
Integration	Testing new features; Integration testing with external systems; Testing new releases	TEST
Production	Live service environment	PROD

19.3.1.5 Role definitions

Role	Tasks	Notes
Admin	Full access	Full access
Developer, including Tester	Writing and maintaining code, submitting pull requests for bugs or features. Creating and running tests on application features.	Developers can write in DEV, but only read in TEST and PROD.
Operations / infrastructure	Manage Terraform modules and infrastructure-as-code deployments. Monitor system health. Apply security controls.	In TEST, PROD, and MGMT, users typically have Read-Only access, with eoRW for critical tasks.
Support and Billing	Monitor usage spend across environments. Read-only access.	

19.3.1.6 Access key

- **RO** - Read Only
- **RW** - Read/Write
- **eRW** - Elevated Read/Write
- **eoRW** - Elevated (optional approval) Read/Write

Note: UserRole refers to the staff role, not the AWS IAM role. Staff may assume multiple roles as needed.

Elevated access method: AWS TEAM (Temporary Elevated Access Management)[43]

43. https://nhsd-confluence.digital.nhs.uk/spaces/AWS/pages/1002780054/AWS+TEAM+Temporary+Elevated+Access+Management

19.3.1.7 Role access matrix by environment

User role	Dev	Test	Prod	Mgmt	Notes
Admin	eRW	eRW	eRW	eRW	Admins have full access to all environments, with elevated access where required.
AWSAdministratorAccess	RW	RW	RW	RW	Admins have full access to all environments.
Developer	RW	RO/eoRW	RO	RO	Developers can write in DEV, but only read in TEST and PROD.
Infrastructure	RW	RO/eoRW	RO/eoRW	RO/eoRW	In TEST, PROD, and MGMT, users typically have Read-Only access, with eoRW for critical tasks.
Support and Billing	RO	RO	RO	RO	Read-Only access for monitoring and billing.

19.3.1.8 Role-based access to AWS services

Each role has access to AWS services across Compute, Data, Networking, Security, Monitoring, and Management. These are suggestions to be scoped at build time.

Role	Compute	Data	Networking	Security	Monitoring	Management
Admin	Full access	Full access	Full access	Full access	Full access	Full access
Developer	Lambda	DynamoDB, S3	VPC, Route 53, CloudFront, Certificate Manager	WAF	CloudWatch, X-Ray	API Gateway, Secrets Manager, Billing Console, Cost Explorer

Role	Compute	Data	Networking	Security	Monitoring	Management
Infrastructure	Lambda	DynamoDB, S3	VPC, Route 53, CloudFront, Certificate Manager	IAM, KMS, CloudTrail, Security Hub, GuardDuty, Amazon Inspector, WAF, AWS Firewall Manager	CloudWatch, X-Ray	API Gateway, CloudFormation, Trusted Advisor, Health Dashboard, Resilience Hub, Well-Architected Tool, IAM Identity Centre, Secrets Manager, Billing Console, Cost Explorer
Support	-	-	-	Trusted Advisor	CloudWatch	Support Centre
Billing	-	-	-	-	-	Billing Console, Cost Explorer

19.3.1.9 Permission sets

Permission sets are role templates used in IAM. A permission set has a collection of permissions specific to a role and environment, mapped to AD groups for user management.

- `SaET-RW-Developer` - Read/Write access for developers in DEV
- `SaET-RO-Infrastructure` - Read-Only access for infrastructure teams in TEST, PROD, and MGMT
- `SaET-RO-Billing` - Read-Only access to billing and cost explorer tools

19.3.1.10 SSO permission sets

Permission sets	Dev	Test	Prod	Mgmt	DR	Notes
SaET-Admin	✅	✅	✅	✅	❌	Full access. DR account restricted for immutable backups.
SaET-RW-Infrastructure	✅	❌	❌	❌	❌	Read/Write infrastructure role for Dev (full access to compute, data).
SaET-RW-Developer	✅	❌	❌	❌	❌	Read/Write developer role for Dev (Lambda, DynamoDB, S3).

Permission sets	Dev	Test	Prod	Mgmt	DR	Notes
SaET-RO-Infrastructure	✗	✓	✓	✓	✗	Read-Only infrastructure role.
SaET-RO-Developer	✗	✓	✓	✓	✗	Read-Only developer role.
SaET-RO-Billing	✓	✓	✓	✓	✗	Read-Only billing role for cost visibility.
SaET-RO-Support	✓	✓	✓	✓	✗	Read-Only support role.

19.4 Options considered

- **Option 1 - Centralized SSO and Terraform-managed permission sets (chosen)**
  - Pros: Consistent access control, auditable changes, least privilege by design.
  - Cons: Requires discipline to keep roles and policies up to date.
- **Option 2 - Per-account IAM roles managed manually**
  - Pros: Fast to start in a single account.
  - Cons: Hard to audit, inconsistent permissions, higher operational risk, difficult to keep track of.
- **Option 3 - Broad admin access with ad-hoc elevation**
  - Pros: Lowest short-term overhead.
  - Cons: Violates least privilege, increases security and compliance risk.

19.5 Rationale

- Centralized configuration provides traceability, auditability, and consistent access patterns across environments.
- Environment scoping reduces blast radius and limits risk.
- Least-privilege permission sets align with organizational security requirements.
- Temporary elevated access supports operational needs while preserving governance.

19.6 Consequences

- Access changes must go through Terraform and IAM Identity Centre processes.
- Teams must maintain and review permission sets as services evolve.
- Break-glass access requires TEAM approval and audit trails.

19.7 References

- AWS Single Sign-On with Azure AD Identity Provider[44]
- DOS Permission Sets[45]
- AWS Single Sign On[46]
- AWS TEAM documentation[47]
- AWS Admins SSO Documentation[48]
- Engineering red lines - Cloud-8 (TEAM)[49]

44. https://github.com/NHSDigital/aws-prod-sso-config/tree/main
45. https://github.com/NHSDigital/aws-prod-sso-config/tree/main/terraform/programmes/DOS
46. https://nhsd-confluence.digital.nhs.uk/spaces/AWS/pages/529502405/AWS+Single+Sign+On
47. https://nhsd-confluence.digital.nhs.uk/x/IjXFOw
48. https://nhsd-confluence.digital.nhs.uk/x/xZCPHw
49. https://nhs.sharepoint.com/sites/X26_EngineeringCOE/SitePages/Red-lines.aspx#cloud-8

20 ADR-API-020 - WAF Rules configuration

ADR Number	ADR-098
Name	WAF Rule
Status	REVIEW
Decision	Option 6
Stakeholders	Richard Beggs Afzal Mufti Olusoji Sogbetun Andrew Walker Jack Cullen

20.1 Drivers

We have requirements to ensure that our APIs are resilient to different types of attack from different attack vectors. Typically we want defence against 'who' is calling our APIs and 'what' they are sending. The 'who' part is often handled by Authentication mechanisms, the 'what' (and 'who') can be handled by a WAF.

20.2 Assumptions

- No assumptions

20.3 Decision

Option 6 - Apply current live rule set apart from the Non GB origin rule as this rule blocks the public github runners that we use to send traffic through to perform tests through the environments. We will implement the AWS BOT Blocking rule, but we will add an exception to this rule that allows the APIM healthcheck ping through by allow listing the APIGEE static IPs.

Consequences: A risk for the business to be aware of in that a change to the static IP hosting of the Apigee platform would result in a total loss of service.

20.4 Options considered

Option 1 - No WAF

In this option there is no WAF implemented at the backend.

- Pro: Nothing to do
- Pro: Authentication mechanisms at Proxy (Private JWT) and Custom Domain (mTLS), already protect us from unknown actors unless circumvented.
- Pro: Rate limiting at Proxy and API Gateway (once past authentication) already protect us from DOS attacks
- Con: Leaves several vectors open to attack. Although behind Private JWT authentication via the Proxy, and mTLS authentication for direct invocations via the API Gateway Custom Domain (DNS), we are missing a defence layer over what an authenticated actor can send through to our APIs. The 'actor' could be a known Connecting Party, or could be an unknown actor that has managed to circumvent either the Private JWT authentication mechanism at the Proxy OR the mTLS authentication mechanism at the Custom Domain of the API Gateway.
- Con: WAF is relatively cheap in the scheme of things, and offers a hard block again malicious know hosts and known injection attacks
- Risk: Should an actor circumvent either authentication mechanism, we are down to relying on explicitly coding in validation checks on what we are receiving. Although it is best practise to do this anyway (and we have done), malicious traffic from known malicious hosts could update/ corrupt data.
- Cost: No cost

Option 2 - WAF with identical rule set to current Live

This option is pretty much a no-go from the off, but is mentioned here for completeness. Adopting the current Live rule set blocks the healthchecks of the API-M platform (Apigee), rendering our service unreachable for connecting parties.

- Pro: None
- Con: No service
- Risk: N/A
- Cost: N/A

Option 3 - WAF with differing rule set to Live, favouring strict host IP rules in the non-development environments.

In this option the WAF will be configured to block all traffic apart from traffic coming from a known source. For the non-development environments, the known sources are:

- API-M / Apigee - identifiable by one of two static IPs

In the development environment, we have the above 2 known sources, and also traffic that is sent via a github runner for running tests directly into the Custom Domain of the API Gateway. Therefore in the development environment, the WAF would be configured to block only known hostile IPs.

The WAF will also contain the following rules to prevent malicious inputs coming through from the known sources:

- AWS Managed Known Bad Input
- AWS Managed Common Rule Set
- AWS Managed Rules Linux Rule Set
- AWS Managed Rules Unix Rule Set

This option provides tight control over what traffic we allow through from where, and also protects us from known malicious inputs from known sources.

- Pro: All traffic at the WAF unless it is from a known IP, preventing BOTs and known hostile hosts from reaching our service. Since all traffic through comes through API-M (Apigee), we would have traceability of the originating source that sent traffic through the proxy.
- Pro: We are protected against known malicious injection / request tampering attempts from known sources.
- Con: There is a difference in rule set up between the development and non-development environments to allow traffic through from github runners in order to send requests straight to the Custom Domain of the API Gateway for our development tests to run in the pipelines. (Although the Custom Domain of the API Gateway is protected by mTLS which would need negotiating before reaching the WAF).
- Con: In the event that the static IPs for Apigee change, we would have a total loss of service until the replacement IPs are allow-listed in the WAFs in the non-development environments.
- Con: It was highlighted in further discussions that github runners would also need to send traffic into the higher environments for the automated tests. This cannot be sensibly done unless we host our own runners.
- Risk: Total loss of service should the Apigee static IPs change.
- Cost: Negligible (couple hundred per month)

Option 4 - WAF with differing rule set to Live, favouring blocks from known malicious IPs / Areas

In this option the WAF will be configured to block only traffic from known malicious IPs and areas by implementation of the following rules:

- AWS Managed IP Reputation List
- Non GB Block (current live bespoke rule)
- Hostile Country Block (current live bespoke rule)

In the development environment, because the github runners send traffic through the Custom Domain of the API Gateway for our pipeline tests, the WAF will NOT implement the Non GB Block rule as this would block traffic from the runners.

The WAF will also contain the following rules to prevent malicious inputs coming through from the known sources:

- AWS Managed Known Bad Input
- AWS Managed Common Rule Set
- AWS Managed Rules Linux Rule Set

- AWS Managed Rules Unix Rule Set

This option provides tight control over what traffic we allow through from where, and also protects us from known malicious inputs from known sources.

- Pro: Traffic is blocked from known malicious IPs and geographical areas.
- Pro: We are protected against known malicious injection / request tampering attempts.
- Pro: There are no static IPs to allow list for Proxy → Backend traffic flows.
- Con: There is a difference in rule set up between the development and non-development environments to allow traffic through from github runners in order to send requests straight to the Custom Domain of the API Gateway for our development tests to run in the pipelines. (Although the Custom Domain of the API Gateway is protected by mTLS which would need negotiating before reaching the WAF).
- Con: Not as locked down as option 3 in terms of only allowing traffic in from known sources.
- Risk: Greater potential from attacks from unknown sources, although they would need to circumvent either the Private JWT authentication at the Proxy level, or the mTLS negotiation at the Custom Domain of the API Gateway.
- Cost: Negligible (couple hundred per month)

Option 5 - Similar to Option 3, but allow listing the github runners in the development environment

In this option all environments would adopt the same set of rules with the exception of the development environment which would have an additional rule to allow list the IPs of the github runners.

- Pro: Better consistency and alignment of rules through the environments
- Pro: Better security for the development environment and the principle of only allowing traffic through from known sources holds.
- Cons: Maintenance effort involved in maintaining the list of github runner IPs
- Risks: None
- Cost: Negligible for the WAF, but a maintenance cost has to be factored in for keeping the github runner IP allow-list maintained.

Option 6 - Apply current live rule set apart from the Non GB origin rule as this rule blocks the public github runners that we use to send traffic through to perform tests through the environments. We will implement the AWS BOT Blocking rule, but we will add an exception to this rule that allows the APIM healthcheck ping through by allow listing the APIGEE static IPs.

For this option, the rules are therefore:

- AWS Managed IP Reputation List
- AWS Managed Bot Control Rule (with the exception of the APIGEE static IPs to allow the API-M healthcheck ping through; and Kainos IPs in the Development environment only if required)
- AWS Managed Known Bad Input
- AWS Managed Common Rule Set

- AWS Managed Rules Linux Rule Set
- AWS Managed Rules Unix Rule Set

Pros and Cons:

- Pro: Traffic is blocked from known malicious IPs.
- Pro: We are protected against known malicious injection / request tampering attempts.
- Pro: We are protected against BOT attacks, and have an exception rule in place for the APIGEE static IPs so that the API-M healthcheck ping is not blocked by this rule.
- Pro: (Development account only: we will allow list Kainos IPs for developers, if required).
- Pro: Better immutability between all environments.
- Con: Not as tight as only allowing traffic through from known hosts, but this option is not feasible unless we migrate over to using Self-hosted github runners.
- Risk: Total loss of service should the Apigee static IPs change.
- Cost: Negligible (couple hundred per month)

20.5 Rationale

20.5.1 Criteria

- Cost - how much is the development / licensing / ongoing cost?
- Effort - how long will it take?
- Maintenance - what is the effort in maintaining the option
- Security Posture - how secure the option is in relation to the API architecture
- Suitability - how suitable the option is in relation to the API architecture

20.5.2 Scoring

	Cost	Effort	Maintenance	Security Posture	Suitability
Option 1 - No WAF	None	No Effort	No effort	Low	Low
Option 2 - WAF with same rule set as current Live	N/A	N/A	N/A	N/A	Not suitable
Option 3 - WAF with differing rule set to current Live, favouring strict host IP rules in the non-development environments	Low	Low	Low	High	Med

Option 4 - WAF with differing rule set to Live, favouring blocks from known malicious IPs / Areas	Low	Low	Low	Medium	High
Option 5 - Similar to Option 3, but allow listing the github runners in the development environment	Low/Med	Low/Med	Low/Med	High	High
Option 6 - Current Live rule set without Non-GB restriction and APIGEE IP exception rule for AWS BOT Control	Low	Low	Low	Med/High	High

20.5.3 Refenerence

ADR-098 - WAF Rules Configuration for FtRS APIs[50]

50. https://nhsd-confluence.digital.nhs.uk/spaces/FRS/pages/1313512404/ADR-098+-+WAF+Rules+Configuration+for+FtRS+APIs

21 ADR-API-021 - Clinical Release Ingestion Shared File Transfer

ADR Number	ADR-021
Name	Clinical Release Ingestion Shared File Transfer
Status	REVIEW
Decision	Option 2
Stakeholders	Richard Beggs Afzal Mufti Olusoji Sogbetun Andrew Walker

21.1 1 Context

NHS Pathways requires a reliable, scalable and auditable way to transfer PAT clinical release data files to downstream consumers.

The release data is currently assumed to be provided as structured files, typically CSV, which must be validated, versioned and transferred in a controlled way. The transfer mechanism must reduce manual handling and provide a clear audit trail for governance, clinical safety and technical assurance.

The preferred long-term approach is to use a shared Pathways Release Service so that release files are published once and consumed consistently by approved consumers.

21.2 1.1 Options

21.2.1 1.1.1 Option 1 - Retain the Current Manual Transfer Approach

Continue with manual or bespoke file transfers between teams and systems.

Rejected:

- Maintains a high operational burden.
- Increases the risk of version errors, partial loads and manual handling mistakes.
- Makes auditing and traceability harder.
- Does not provide a clear shared integration pattern.

21.2.2 1.1.2 Option 2 - Shared Pathways Release Service and Landing Bucket (RECOMMENDED)

PAT provides a dedicated S3 bucket and restricted upload location within its AWS account.

PAT publish the clinical release files into the dedicated S3 bucket.

Patient triage Api assumes a dedicated cross-account IAM role using AWS STS temporary credentials and downloads the clinical date release file directly from the bucket.

Pros

- Provides an auditable and reproducible release handoff.
- Clear ownership of the integration boundary.
- Reduces manual intervention and the risk of version contamination.
- Supports monitoring, alerting and rollback of failed loads.
- Allows future consumers to reuse the same release delivery pattern.
- Pathways retains control of storage, retention, monitoring and processing.
- Full auditability through AWS CloudTrail.

Cons

- Introduces a file-based integration rather than a synchronous request/response pattern.
- Requires coordination on file format, versioning.
- Requires AWS IAM configuration between accounts.
- Needs agreed operational handling for failed uploads or incomplete releases.

21.2.2.1 1.1.2.3. Implementation Option 1 - Bucket Actions (RECOMMENDED)

Event triggers applied to the S3 bucket allow for a lambda to be triggered when a new item is created in the S3 bucket.

21.2.2.1.1 1.1.2.3.1. Pros

- Event based (triggers when required)
- Simple
- Cheap
- Limited monitoring and filtering of files

21.2.2.1.2 1.1.2.3.2. Cons

- Limited to what actions can be performed on an S3 bucket.

21.2.2.2 1.1.2.4. Implementation Option 2 - AWS Event Bridge

AWS Event Bridge configured to monitor additions to the S3 bucket and trigger a lambda when required (i.e. the file is uploaded).

21.2.2.2.1 1.1.2.4.1. Pros

- Event based (triggers when required)
- More flexible / expandable than Implementation Option 1
- Better monitoring options

21.2.2.2.2 1.1.2.4.2. Cons

- More complex
- Flexibility not required
- Event bridge comes with a cost per action

21.2.2.3 1.1.2.5. Implementation Option 3 - Scheduled Task

A service is scheduled to check the bucket periodically and perform the processing.

21.2.2.3.1 1.1.2.5.1. Pros

- Portable
- Simple

21.2.2.3.2 1.1.2.5.2. Cons

- Runs on a schedule, so may run when not required.
- Delays to uploading data would result in missed data unless redundancy is implemented (inefficient)

21.2.3 1.1.3. Option 3 - Patient Triage API-Owned S3 Bucket

The Patient Triage API provides a dedicated S3 bucket and restricted download location within its AWS account. PAT uploads the service file to this as part of their release process.

PAT assumes a dedicated cross-account IAM role using AWS STS temporary credentials and downloads the service file from the bucket before processing.

21.2.3.1 1.1.3.1. Pros

- Uses AWS-native authentication through IAM role assumption.
- Clear ownership of the integration boundary.

21.2.3.2 1.1.3.2. Cons

- Requires AWS IAM configuration between accounts.
- Resolution may lie with either the Patient Triage team or the PAT team.
- Less control over implementation than Option 2.

- PAT would be responsible for entire delivery instead of supporting access.

21.2.4 1.1.4. Option 4 - Consume a PAT API

PAT make the clinical release files available via an API endpoint which the patient triage Api consume.

21.2.4.1 1.1.4.1. Pros

- Data may be accessed by other services.

21.2.4.2 1.1.4.2. Cons

- No control over timescales.
- No control over what the API outputs.
- Would require a PAT service to build an API service.
    - Inefficient

21.2.5 1.1.4 Decision

Implement Option 2 - Shared Pathways Release Service and Landing Bucket.

PAT to publish clinical release files into a S3 bucket. Approved consumers will consume files from that location using agreed access controls, file standards and governance controls.

21.2.6 1.1.5 Consequences

21.2.6.1 Positive

Establishes one shared and repeatable transfer pattern for clinical release files. Improves traceability, governance and operational control. Reduces manual processing and the risk of incomplete or inconsistent transfers. Supports clinical safety and technical assurance by making releases reproducible and reversible. Keeps the design flexible as consumer implementations evolve.

21.2.6.2 Negative

Introduces a file-based delivery model rather than live integration. Requires agreed ownership of the shared transfer mechanism and supporting operational processes. Needs clear handling for schema changes, failed uploads and late or missing files.

21.2.7 References

ADR 002 - Retire the DoS Service Update API Endpoint[51]

51. https://nhsd-confluence.digital.nhs.uk/spaces/NP/pages/1399700637/ADR+002+-+Retire+the+DoS+Service+Update+API+Endpoint

22 ADR-API-022: Performance testing with Jmeter

ADR Number	ADR-API-022: Performance testing with Jmeter
Name	Performance testing with Jmeter
Status	REVIEW
Decision	Option 1
Stakeholders	Richard Beggs Afzal Mufti Olusoji Sogbetun Andrew Walker

22.1 Context

The team needs a repeatable, low-operations approach to execute Apache JMeter performance tests in AWS.

Current pain points:

- Running AWS performance tools is inconsistent and hard to reproduce.
- Test logs and artifacts are not centrally stored.
- Test results appear to be not as accurate as other methods.

Constraints and requirements:

- Prefer managed services to minimize host/cluster operations.
- Support scheduled and on-demand test runs.
- Persist test artifacts (JTL, logs, HTML reports) with retention.
- Support ad hoc invocation by authorized users.
- Use IAM-based access and avoid hardcoded credentials.

22.2 Decision

We will run JMeter tests in containerized tasks on Amazon ECS using Fargate capacity.

Key design points:

- Package JMeter in a Docker image stored in Amazon ECR.
- Store test plans and test data in Amazon S3.

- Orchestrate runs with AWS Step Functions.
- Persist artifacts and reports to Amazon S3.
- Evaluate results against agreed SLO thresholds and record outcome in a run summary.

22.3 Decision Drivers

- Lower operational overhead through serverless container capacity.
- Fast scalability for parallel/distributed load tests.
- Native integration with AWS IAM, CloudWatch, S3, and pipeline services.
- Improved traceability and auditability of test outcomes.
- Better alignment with the tech radar.

22.4 Considered Options

1. ECS on EC2
2. ECS Fargate (chosen)
3. EKS (Kubernetes)
4. Dedicated external load-testing platform
5. Local/self-hosted runners

22.4.1 Option 1: ECS on EC2

Run containerized JMeter workloads on an Amazon ECS cluster backed by self-managed EC2 instances, giving the team direct control over host sizing, scaling, and operating-system-level tuning for heavier or longer-running performance tests.

Pros:

- Lower unit cost at sustained high usage.
- More host-level tuning control.
- Supports tailored worker sizing and capacity planning for heavier or longer-running tests.

Cons:

- Requires instance provisioning, patching, autoscaling, and capacity planning.
- Higher operational burden than Fargate.

22.4.2 Option 2: ECS Fargate (chosen)

Run containerized JMeter workloads on Amazon ECS using serverless task capacity managed by AWS, reducing the need to manage host infrastructure while still allowing the team to package, schedule, and scale test execution through standard ECS task definitions and AWS-native integrations.

Pros:

- Serverless container execution with minimal infrastructure ops.
- Straightforward scaling by task count.
- Native integration with AWS monitoring and security controls.

Cons:

- Potentially higher cost than EC2 for long-running high-volume workloads.
- Task-level resource limits require test sizing discipline.

22.4.3 Option 3: EKS

Run JMeter workloads on Amazon Elastic Kubernetes Service using Kubernetes for orchestration, scaling, and workload management, which can be attractive if the organization already operates Kubernetes and wants to align performance testing with the same platform and operational model.

Pros:

- Strong fit if Kubernetes platform already exists.
- Flexible workload orchestration.

Cons:

- Higher platform complexity for teams not already operating Kubernetes.

22.4.4 Option 4: External platform

Use a third-party managed load-testing product instead of building and operating the execution platform in AWS, shifting more of the execution, orchestration, reporting, and operational support responsibility to an external vendor in exchange for less internal platform ownership.

Pros:

- Reduced internal implementation effort.

Cons:

- Vendor lock-in, licensing cost, and potentially weaker AWS-native integration.

22.4.5 Option 5: Local/self-hosted runners

Run JMeter from developer machines or internally managed runners outside a formal AWS orchestration platform, providing a simple way to execute tests quickly but with less consistency, governance, scalability, and central visibility than a managed cloud-based execution model.

Pros:

- Fast initial setup.

Cons:

- Poor repeatability, weak governance, and limited scalability.

22.5 Consequences

Positive:

- Consistent, repeatable performance test execution.
- Centralized logs, metrics, and artifacts for troubleshooting and trend analysis.
- Flexible execution on demand without blocking deployment pipelines.
- Better security posture using IAM roles and managed secret stores.
- Reduced cluster/host management burden by using serverless task capacity.

Negative:

- Initial setup effort for image build, orchestration and metric extraction.
- Need to design distributed test strategy to avoid generator bottlenecks.
- Ongoing responsibility for instance patching, scaling policy, and AMI lifecycle management.
- Potentially higher cost than EC2 at sustained, high-volume runtime.
- Must design tests within Fargate task resource limits and quotas.

22.6 Architecture (High-Level)

1. An authorized user triggers an ad hoc performance test workflow.
2. Step Functions starts one or more ECS tasks on Fargate capacity.
3. Tasks execute JMeter in non-GUI mode.
4. Results and reports are uploaded to S3.
5. Existing application metrics are reviewed against SLO thresholds.
6. A run summary is generated for manual review and sign-off.

22.7 Security and Compliance

- Use ECS task roles for S3, and Secrets Manager access.
- Store credentials in AWS Secrets Manager or SSM Parameter Store.
- Run Fargate tasks in private subnets where applicable.
- Apply least-privilege IAM policies and network controls for task-level isolation.

22.8 Cost Considerations

- Primary cost drivers: Fargate vCPU/memory runtime, optional ephemeral storage, S3 storage, data transfer.
- Mitigations:
- Right-size task CPU/memory and use efficient test durations.
- Run heavy tests on approved ad hoc windows.

- Apply S3 lifecycle and CloudWatch retention policies.

22.9 Implementation Plan

1. Build and publish JMeter container image to ECR.
2. Create an ECS cluster and Fargate task definition.
3. Create S3 buckets/prefixes for test plans and artifacts.
4. Implement Step Functions workflow for run orchestration.
5. Add metric extraction and threshold evaluation step.
6. Expose manual invocation entry points (AWS Console).
7. Pilot with one service, validate thresholds, then scale adoption.

22.10 Acceptance Criteria

- Test runs can be triggered on demand by authorized users.
- Logs are visible in CloudWatch / Grafana for all test runs.
- JTL and HTML report artifacts are persisted in S3.
- Run summaries clearly indicate threshold pass/fail status for manual decision-making.
- Historical run data is available for trend comparison.

23 ADR-API-023: Green-Blue Deployment Options

ADR Number	ADR-API-023: Green-Blue Deployment Options
Name	Green Blue Deployment Options
Status	APPROVED
Decision	Option 1
Stakeholders	Richard Beggs Afzal Mufti Nitin EXTERNAL-shukla Andrew Walker

23.1 Context

The SAET Patient Triage API is deployed on AWS API Gateway with multiple Lambda function versions (v1, v2) serving FHIR-versioned traffic routed via Accept header matching (application/fhir+json; version=1 and version=2).

To enable a safe promotion of new application releases with zero downtime, the platform requires a deployment strategy that minimises risk to live NHS client traffic.

23.1.1 Constraints and requirements:

A key constraint is that client systems, including APIM proxies, have a single consistent API URL configured in their proxy configuration (api.patient-triage.service.nhs.uk[52]).
This URL is not expected to change between deployments and must always resolve to the same TLS endpoint, with mTLS authentication enforced at the API Gateway layer.

The platform is therefore required to provide a green/blue deployment mechanism that meets the below requirements;

- Maintains a single, stable client-facing URL throughout all deployment phases.
- Preserves mTLS termination at the API Gateway custom domain level.
- Allows the new deployment to be tested end-to-end before receiving live traffic.
- Supports fast rollback in the event of a failed deployment.
- Minimises operational complexity and additional infrastructure cost.

52. http://api.patient-triage.service.nhs.uk/

23.2 Decision Drivers

23.2.1 Key design points:

- Single consistent client URL i.e. client has api.patient-triage.service.nhs.uk[53] configured in APIM proxy settings.
- mTLS enforcement i.e. client certificate authentication must be maintained at the API Gateway layer.
- Pre-production validation i.e. new deployments must be testable in isolation before receiving live traffic.
- Fast rollback i.e. ability to revert to the previous deployment version rapidly if issues arise post-switchover.
- Operational simplicity i.e. minimise platform complexity and additional infrastructure overhead.
- Cost efficiency i.e. avoid unnecessary additional AWS service costs.
- No DNS propagation risk i.e. switchover must not introduce client-side DNS caching issues.

23.3 Considered Options

1. API Gateway Routing Rule Amendment (Proposed)
2. Application Load Balancer (ALB) with Weighted Target Groups
3. API Gateway Stages for Blue/Green
4. Route 53 Weighted Routing Records
5. Routing Lambda

23.3.1 Option 1: API Gateway Routing Rule Amendment (Proposed)

Two separate API Gateway deployments (green and blue) are maintained, each with their own colour-coded custom domain names (green-prod.patient-triage.service.nhs.uk[54] and blue-prod.patient-triage.service.nhs.uk[55]).
A third main custom domain (api.patient-triage.service.nhs.uk[56]) is created as the single client-facing endpoint.
API Gateway routing rules on the main custom domain are amended at switchover time to point to the newly validated API Gateway deployment.
Each deployment independently handles FHIR Accept header versioning internally, routing version=1 and version=2 traffic to the appropriate Lambda function version.

For example, a switchover to blue environment is performed via a single Terraform variable change below;

53. http://api.patient-triage.service.nhs.uk/
54. http://api.patient-triage.service.nhs.uk/
55. http://api.patient-triage.service.nhs.uk/
56. http://api.patient-triage.service.nhs.uk/

```
terraform apply -var="active_deployment=blue"
```

Since the switchover is an internal AWS API Gateway control plane operation, the Route 53 record for api.patient-triage.service.nhs.uk[57]never changes. There is no DNS propagation delay and no client-side cache impact.

Pros

- Single consistent client URL which never changes
- mTLS terminated at API Gateway main custom domain
- No DNS propagation delay. Instantaneous switchover
- New deployment testable via dedicated domain pre-switchover
- Instant rollback via single Terraform apply
- Minimal additional AWS service cost
- Route 53 record immutable throughout lifecycle

Cons

- All-or-nothing traffic switch. API GW routing rule does not support percentage-based splitting.
- Terraform state lock could temporarily block switchover whilst another pipeline is running.
- Requires three custom domain names to manage with ACM certificate covering all three domains.
- Truststore must be maintained consistently across all three custom domains (main, green and blue).
- API Gateway routing rule propagation has a small internal AWS delay (a few seconds)

23.3.2 Option 2: Application Load Balancer (ALB) with Weighted Target Groups

An Application Load Balancer (ALB) sits between Route 53 and the two API Gateway deployments (green and blue), providing weighted traffic routing between them.
Route 53 points to the ALB, which forwards traffic to blue or green based on configured target group weights.

Pros

- Native percentage-based traffic splitting.
- Single client URL maintained.
- Gradual canary rollout possible without DNS changes.

Cons

- ALB cannot natively target API Gateway custom domain names directly i.e. it requires complex dynamic IP management involving multiple VPC endpoint IP addresses etc.

57. http://api.patient-triage.service.nhs.uk/

- mTLS conflict i.e. ALB terminates TLS before API Gateway, preventing client certificates from reaching API Gateway's truststore validation.
- Double mTLS required i.e. ALB authenticates client, then ALB must authenticate to API Gateway separately.
- Significant additional infrastructure complexity required.
- Additional costs associated with ALB hourly charge plus LCU costs.
- Does not align with existing API Gateway-centric architecture.

23.3.3 Option 3: API Gateway Stages for Green/Blue

Green and blue deployments are represented as separate stages within a single API Gateway (e.g. stage: blue, stage: green).
Traffic is switched by updating the base path mapping or routing rules to point to the active stage.

Note: API Gateway stages are already in use in the Patient Triage API platform for Lambda function versioning (v1, v2).
Reusing stages for green/blue introduces a naming collision and couples two orthogonal concerns i.e. Lambda version routing and deployment promotion within the same construct.

Pros

- No additional API Gateway deployments required.
- Simpler initial setup.
- No additional cost.

Cons

- API Gateway stages already used for Lambda function versioning. Coupling green/blue to API Gateway stages conflicts with existing design.
- No pre-switchover isolation. Green and blue share the same API Gateway, limiting independent testing.
- Rollback requires re-deployment rather than a routing switch.
- Increases risk of misconfiguration affecting both environments simultaneously.
- Does not provide clean separation of deployment environments.

23.3.4 Option 4: Route53 Weighted Routing Records

Two Route 53 weighted records are created for api.patient-triage.service.nhs.uk[58]i.e. pointing to green and blue custom domain names respectively.

Traffic is then split between green and blue by adjusting the Route 53 record weights.

Pros

- Native percentage-based traffic splitting
- No API Gateway changes required during switchover

58. http://api.patient-triage.service.nhs.uk/

- Familiar AWS-native mechanism

Cons

- APIM have server URL api.patient-triage.service.nhs.uk[59]configured in proxy. DNS-level splitting between green and blue exposes clients to different TLS endpoints, breaking the single-domain contract.
- mTLS termination is split across two custom domains i.e. green and blue with each independently maintaining truststore and ACM certificate configuration.
- DNS propagation delay expected with changes taking approximately 30–60 seconds or longer to propagate. Clients with cached DNS records may continue hitting the old deployment.
- TTL management required to minimise propagation windows.
- Rollback subject to the same DNS propagation delay.

23.3.5 Option 5: Routing Lambda

We are aware APIM are/were looking at a router option at the Apigee proxy level.

https://nhsd-confluence.digital.nhs.uk/spaces/APM/pages/171844371/D038+-+Blue-Green+deploy+process+...[60]

Having said that, a routing lambda option for splitting traffic has also been considered on a high level as part of the blue green deployment on our part i.e. having a routing lambda placed between the main API gateway custom domain and the blue/green API gateway deployments.

Pros

- Allows percentage-based traffic splitting.
- No API Gateway changes required during switchover.
- No DNS propagation delay.

Cons

- Introduces additional latency per request i.e. the extra lambda invocation is expected to add more time per request on top of the existing response time.
- Introduces risk of single point of failure for all traffic i.e. larger failure blast radius. The routing lambda sits on the critical pat with a bug or cold start affecting all traffic.
- Additional costs for the extra routing lambda invocations and putting mitigation in place for potential cold start risk.
- Higher operational and architectural complexity including the proxying of requests and responses, two API gateway hops etc.

59. http://api.patient-triage.service.nhs.uk/
60. https://nhsd-confluence.digital.nhs.uk/spaces/APM/pages/171844371/D038+-+Blue-Green+deploy+process+for+API+Proxies

23.4 Comparison Matrix

The following matrix compares all options against the key decision drivers identified above.

Criterion	API GW Routing Rule (Proposed)	ALB	API GW Stages	Route 53 Weighted	Routing Lambda
Single consistent client URL	✅ Yes	✅ Yes	✅ Yes	✅ Yes	✅ Yes
mTLS termination at entry	✅ API GW	⚠️ ALB only	✅ API GW	⚠️ Split	✅ API GW
Gradual traffic splitting	❌ No	✅ Yes	❌ No	✅ Yes	✅ Yes
Instant rollback	✅ Yes	⚠️ Partial	❌ Re-deploy	❌ DNS delay	✅ Yes
Pre-switchover testing	✅ Yes i.e. via dedicated domain	✅ Yes	⚠️ Limited	⚠️ Limited	✅ Yes i.e. via dedicated domain
Infrastructure complexity	Low	High	Medium	Medium	High
Additional cost*	None	ALB hourly	None	None	Double Invocations
DNS propagation risk	❌ None	❌ None	❌ None	⚠️ Yes	❌ None
Terraform simplicity	✅ Simple	❌ Complex	⚠️ Medium	⚠️ Medium	❌ Complex
Latency Impact	None	Minimal	None	None	Additional ms per request
Works with Lambda versioning	✅ Yes	✅ Yes	⚠️ Coupled	✅ Yes	✅ Yes i.e. via API GW
Single point of Failure	Low	Low	Low	Low	❌ High

*excludes additional costs associated with having separate blue and green environment AWS components

✅ Satisfies criterion ⚠️ Partial / conditional ❌ Does not satisfy

23.5 Decision

Option 1 : API Gateway Routing Rule Amendment is selected as the green/blue deployment strategy for the SAET Patient Triage API.

This option is the only approach that satisfies all non-negotiable constraints simultaneously (a single consistent client URL, mTLS termination exclusively at API Gateway, no DNS propagation risk, pre-production isolation via dedicated colour-coded domains, and instant rollback capability) i.e. all without introducing additional infrastructure cost or complexity.

The all-or-nothing traffic switch is accepted as a deliberate trade-off, mitigated by the following controls;

- Full end-to-end testing of the new deployment via its dedicated non-live leg prior to switchover.
- Switchover scheduled during a low-traffic window if possible.
- ODIN dashboards and logs monitored for a minimum of 15 minutes post-switchover.
- Previous deployment (green) retained live on its dedicated domain as an instant rollback target should the need arise.
- Rollback executable via single Terraform apply.
- Break glass TEAM access to PROD AWS console is also available for making the straight forward API Gateway routing rule switch should there be an issue with terraform state and rollback is needed urgently.

23.6 Consequences

23.6.1 Positive:

- Deployment risk is significantly reduced through pre-production isolation and fast instant rollback.
- Client systems require no reconfiguration during or after deployment.
- mTLS security posture is maintained throughout the deployment lifecycle.
- Infrastructure remains Terraform-native with no additional AWS services.

23.6.2 Negative:

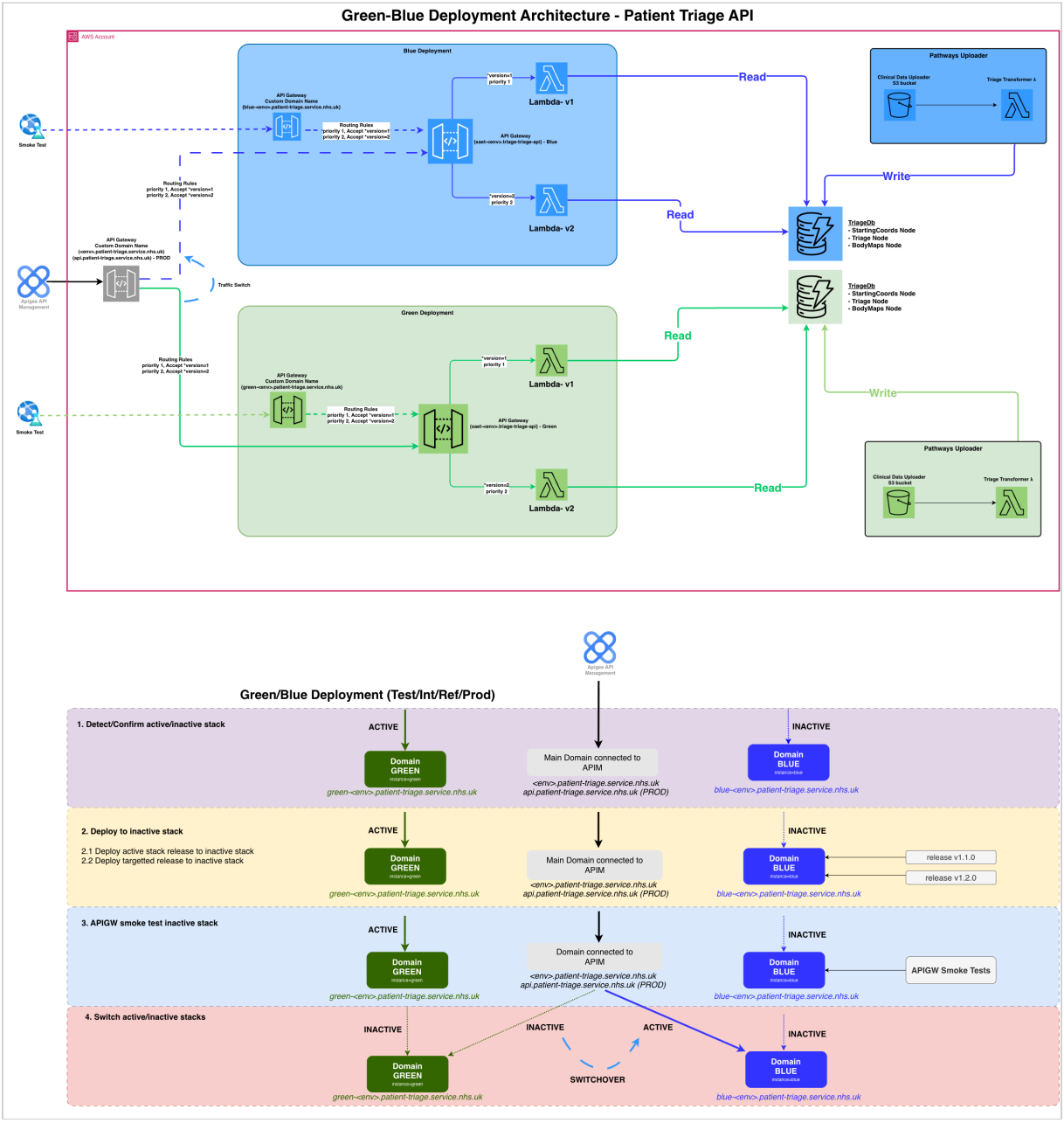
- Three custom domain names with each requiring ACM certificate coverage.
  Wildcard *.patient-triage.service.nhs.uk[61]recommended with subject alternative name (SANs) *.<env>.patient-triage.service.nhs.uk[62] included for the coloured leg domains.
- Truststore must be maintained consistently across blue and green deployments.

- Traffic switchover is all-or-nothing. This is mitigated by pre-switchover validation/testing controls and instant rollback if required.
- Terraform state lock dependency. The API Gateway routing rule switchover can also be carried out via AWS Console using break glass TEAM access should there be a terraform state issue.

23.7 Architecture (High-Level)

The diagram below provides a high-level architecture

61. http://api.patient-triage.service.nhs.uk/
62. http://api.patient-triage.service.nhs.uk/



23.8 Security and Compliance

23.8.1 mTLS Termination at API Gateway

A fundamental security requirement of this architecture is that mTLS terminates exclusively at the API Gateway custom domain layer.

This is not an implementation preference but a deliberate security and compliance control for the following reasons highlighted below;

Zero Trust Perimeter

Terminating mTLS at the outermost boundary of the AWS infrastructure means that unauthenticated or unauthorised requests are rejected before entering any internal network layer.

Truststore as Cryptographic Access Control List

The API Gateway custom domain is backed by an S3-hosted truststore containing the certificates of all authorised calling systems i.e. including NHS APIM client proxies.

Only systems presenting a certificate matching an entry in the truststore are permitted to connect. This access control is enforced at the cryptographic level and cannot be bypassed by application bugs or misconfigured IAM policies.

Separation of Concerns

Terminating mTLS at API Gateway maintains a clean architectural boundary i.e. API Gateway owns transport-layer authentication (mTLS + API key enforcement), Lambda owns business logic and application-layer authorisation. This separation ensures that authentication cannot be bypassed by application-layer changes and simplifies security review and penetration testing scope.

23.8.2 Data in Transit

All traffic between client systems and API Gateway is encrypted via TLS 1.2 minimum, enforced by the API Gateway regional endpoint configuration. Traffic between API Gateway and Lambda functions operates within the AWS internal network and is encrypted by default. No data is transmitted over unencrypted channels at any point in the request lifecycle.

23.9 Cost Considerations

The proposed option introduces minimal additional cost compared to a single-deployment architecture. The primary cost drivers are highlighted in table below;

23.9.1 Proposed Option i.e. API Gateway Routing Rule Amendment

Component	Cost Impact	Notes
Three API GW custom domains	Negligible	No per-domain charge; ACM certificate is free

Component	Cost Impact	Notes
Routing rules (x3 domains)	No additional charge	No charge for routing rules
Second API GW deployment (standby)	REST API charges apply only to requests processed	Standby deployment incurs no cost when not receiving traffic
Second set of Lambda functions (standby)	No cost when not invoked	Lambda charges per invocation; standby Lambdas receive no traffic
Second DynamoDB tables (standby)	Storage cost only	Read/Write capacity charges only apply to active deployment
ACM Wildcard certificate (*.patient-triage.service.nhs.uk[63]) with subject alternative name (SANs) *.<env>.patient-triage.service.nhs.uk[64] included for the coloured leg domains	Free	AWS Certificate Manager public certificates are free
S3 truststore bucket (shared)	Negligible	Single shared truststore bucket, however additional minimal costs for different truststore PEM files associated with each domain
CloudWatch logs — both deployments	Per GB ingested	Green deployment logs during pre-cutover testing; minor additional cost

The standby deployment (green after switchover to blue, or vice versa) is retained as a rollback target.

23.9.2 Rejected Options Cost Comparison

Option	Estimated Additional Cost	Cost Driver
Proposed (Routing Rule)	Minimal	No additional services; standby DynamoDB storage only
ALB	Additional monthly costs incurred	ALB hourly rate (~$0.0084/LCU/hr) plus fixed hourly charge regardless of traffic (~$0.02646/hr); DTO charges

63. http://api.patient-triage.service.nhs.uk/
64. http://api.patient-triage.service.nhs.uk/

Option	Estimated Additional Cost	Cost Driver
API GW Stages	Minimal	No additional cost but architectural coupling risk
Route 53 Weighted	Minimal	Route 53 health check charges if used (~$0.50/check/month); negligible
Routing Lambda	Low to Medium	Double Lambda invocations per request; provisioned concurrency required to mitigate cold start

The ALB and Routing Lambda options are the only approach that introduce meaningful ongoing cost.

Based on expected API traffic volumes, the ALB option would incur additional costs per month/ environment with no architectural benefit over the proposed option and with the additional mTLS complexity.

The Routing Lambda option includes additional architectural / operational complexity, cost and latency without any sufficient benefit over the proposed approach other than the gradual splitting of traffic.
Considering it takes just a couple of seconds for amending API gateway routing rule (all-or-nothing switchover option) during a rollback, this benefit doesn't provide much as a trade-off.

23.9.3 Cost Optimisation

- Use of a single wildcard ACM certificate (*.patient-triage.service.nhs.uk[65]) covering all three custom domains to eliminate per-certificate management overhead
- A single S3 truststore bucket used to store all PEM files associated with all three custom domains.

23.10 Implementation Plan

23.10.1 Pre-Switchover

- Deploy new release to non-live environment and run full end-to-end tests against blue-prod.patient-triage.service.nhs.uk[66]
- Confirm ODIN dashboards, logs show healthy response codes and latency on blue leg.
- Ensure Terraform state is not locked and pipeline is clear to be ran.

65. http://api.patient-triage.service.nhs.uk/
66. http://api.patient-triage.service.nhs.uk/

23.10.2 Switchover

Switchover is to be executed via github pipeline setting `active_deployment` leg colour to the tested non-live environment which runs on a newer released version so as to initiate the API Gateway routing rule change.

`terraform apply -var="active_deployment=blue"`

```
active_api_id    = var.active_deployment == "green" ?
aws_api_gateway_rest_api.triage.id : aws_api_gateway_rest_api.triage_blue.id
active_stage_v1  = var.active_deployment == "green" ?
aws_api_gateway_stage.v1.stage_name : aws_api_gateway_stage.v1_blue.stage_name
active_stage_v2  = var.active_deployment == "green" ?
aws_api_gateway_stage.v2.stage_name : aws_api_gateway_stage.v2_blue.stage_name
```

API Gateway Routing rules on api.patient-triage.service.nhs.uk[67]updated which involves internal AWS control plane operation. There is no DNS change.

23.10.3 Post-Switchover

- Monitor ODIN dashboard, logs and alerts for minimum 15 minutes.
- Confirm client traffic (APIM / Apigee) is routing correctly via main domain.
- Green deployment i.e. the new non-live environment following the switchover is retained as rollback target.

23.10.4 Rollback

Rollback will also be executed via github pipeline by setting `active_deployment` leg colour back to the previous live environment (running on older version) so as to initiate the API Gateway routing rule change i.e.

`terraform apply -var="active_deployment=green"`

```
active_api_id    = var.active_deployment == "green" ?
aws_api_gateway_rest_api.triage.id : aws_api_gateway_rest_api.triage_blue.id
active_stage_v1  = var.active_deployment == "green" ?
aws_api_gateway_stage.v1.stage_name : aws_api_gateway_stage.v1_blue.stage_name
active_stage_v2  = var.active_deployment == "green" ?
aws_api_gateway_stage.v2.stage_name : aws_api_gateway_stage.v2_blue.stage_name
```

67. http://api.patient-triage.service.nhs.uk/